

Understanding Nondeterminism: From Witnesses and Structure to Faster SAT

Samuel Schlesinger

September 5, 2026

Abstract

Nondeterminism asks whether one globally consistent witness makes a deterministic check accept. This deep dive follows that idea from computation paths and verifier relations to shared circuit values, exact counting tables, and structural bounds on eliminating witnesses. We derive a SAT and exact-counting algorithm that beats exhaustive search for single-output Boolean circuits with u inputs and $s \leq (4 - \gamma)u$ gates, for every fixed $0 < \gamma < 4$, over the full binary basis B_2 with unrestricted fan-out. For each fixed $\varepsilon > 0$, its running time is $\text{poly}_\varepsilon(s + u + 1)2^{(1/4+\varepsilon)(s+1)}$, allowing exponential space. The sharper exponent is $\min\{b, (1/3 + \delta)(s + 1 - b)\}$ for fixed $\delta > 0$, where b counts the inputs in a normalized output cone. The proof encodes consistent circuit values as tables, preserves exact counts through cubic reduction, and combines the Fomin–Høie pathwidth theorem with frontier contraction and input enumeration.

The same consistency budget explains when existential projection $g(x) = \exists y f(x, y)$ has a small deterministic circuit. For an s -gate verifier, we prove $C(g) = O_\varepsilon((s + 1)2^{(1/5+\varepsilon)(s+1)})$. We derive necessary structure for near-full amplification at the critical gate budget and realize nearly critical supplied budgets with easy projections, exposing the role of nonminimal verifiers. A joint witness-sampling and shared-computation bound gives further improvements, illustrated by analytical families. Pinning input tables also retains the counting exponent for search, lexicographic rank and unrank, and expected-time exact uniform sampling.

The algorithmic improvement is over exhaustive search on the stated restricted instances. Historical priority and best-known unrestricted-space status of the precise bound remain unestablished. The full counting algorithm is not implemented or benchmarked here; the methods appendix records the AI-assisted workflow and the finite checks of supporting constructions.

Contents

1	From computation paths to shared witnesses	3
2	A complete example: from shared values to counting tables	8
3	SAT and exact counting below four gates per input	12
4	The question and its quantifiers	13
5	The universal ceiling and the first crossover	14
6	Circuit structure yields a second easy range	16
7	A stronger simulation from the consistency graph	18
8	What the algorithm gives beyond a yes-or-no answer	24
9	What the structural bounds do and do not force	27
10	Many witnesses give small nonuniform circuits	31
11	Worked families: what the parameters reveal	35
12	What random verifiers do, and do not, demonstrate	39
13	Unconditional lower bounds and their limits	40
14	A precise conditional version of exponential hardness	41
15	Relation to prior work and the scope of the contributions	42
16	What would sharpen the boundary?	44
A	Weaker aggregation: a further direction	46
B	AI-assisted methods, verification, and reproducibility	48

1 From computation paths to shared witnesses

Suppose a proposed solution can be checked efficiently. A path through a graph, for example, is checked by reading its vertices and checking its successive edges. Checking one proposed path and finding a path are different tasks. Nondeterminism gives a mathematical language for the first task together with an existence condition: an input is accepted when at least one legal proposal passes the check. Understanding that condition is the starting point for both the circuit-size question and the SAT algorithm.

1.1 A computation tree describes possibilities

A deterministic computation has one next step at each nonhalting configuration. A nondeterministic computation may have several legal next steps. Starting from a fixed input, unfold these possibilities into a tree. A root-to-leaf path is one computation, and an accepting leaf records one successful computation. The machine accepts the input if there exists an accepting path; it rejects if all paths reject. In the time-bounded setting used here, every path halts within the specified bound. Its nondeterministic running time measures the longest path, not the total number of tree nodes.

After encoding the finitely many possible transitions by a fixed number of bits, a path can be specified by its successive choices. A deterministic checker receives the original input and this choice record, simulates the selected steps, rejects illegal choices, and checks that the outcome accepts. Conversely, a machine can guess a proposed choice record and run the checker. Thus an accepting path can be replaced by a finite *certificate*, also called a *witness*. A certificate need not literally list machine transitions: a satisfying assignment or a proposed graph path may be a more convenient representation of the same acceptance condition.

This replacement describes existence, without promising an efficient procedure for discovering the certificate. An acceptance proof can exhibit one certificate. A deterministic rejection procedure must justify the absence of every certificate, possibly by exploiting shared structure rather than inspecting them one at a time. Nor does the computation tree provide free physical parallelism: an implementation that assigns one processor to every path must account for those processors, their work, and their storage. The abstract path-length measure does not supply that hardware.

1.2 The verifier relation and its projection

Write $\{0, 1\} = \{0, 1\}$. For the moment fix lengths n and m , and let

$$f : \{0, 1\}^n \times \{0, 1\}^m \longrightarrow \{0, 1\}$$

be a deterministic checker. The string x is the *ordinary input*, which specifies the question; y is the proposed witness. The equation $f(x, y) = 1$ means that this particular proposal is valid. The set of valid witnesses over x , called its *fiber*, is

$$W_x = \{y \in \{0, 1\}^m : f(x, y) = 1\}.$$

Forgetting which witness worked leaves the Boolean function

$$g(x) = \exists y f(x, y) = \bigvee_{y \in \{0, 1\}^m} f(x, y) = \begin{cases} 1, & W_x \neq \emptyset, \\ 0, & W_x = \emptyset. \end{cases} \quad (1)$$

Here an existential assertion is identified with its Boolean truth value. Geometrically, the accepted pairs form a relation inside $\{0, 1\}^n \times \{0, 1\}^m$, and $g^{-1}(1)$ is its projection onto the first coordinate.

Many pairs may project to the same ordinary input. Projection retains whether a fiber is empty and discards its size and internal shape.

We represent a checker by a *Boolean circuit*: gates compute Boolean operations and wires carry their input and output values, with no directed feedback. A gate's *fan-in* counts incoming wires; a value's *fan-out* counts its uses. The *output cone* consists of the vertices with a directed path to the designated output, including that output. Vertices outside it can be discarded. An input is *essential* if flipping it changes the function on some assignment of the other inputs; merely occurring in the output cone does not guarantee this property.

Object	Operational meaning
$x \in \{0, 1\}^n$	The instance held fixed while proposals are checked.
$y \in \{0, 1\}^m$	One complete proposal, shared by every check that uses it.
$f(x, y)$	The deterministic answer for that instance and proposal.
W_x	All successful proposals for the fixed instance.
$g(x)$	Whether there is at least one successful proposal.
s	Number of gates in a supplied verifier circuit, when one is given.
$C(g)$	Minimum number of gates in any circuit computing g .

Table 1: The same relation viewed as verification, existence, and circuit size. The gate model is specified below; a supplied size need not be minimum.

A running example. Consider the three-gate checker

$$v_1 = x \vee y, \quad v_2 = y \oplus z, \quad f(x, y, z) = v_1 \wedge v_2, \quad (2)$$

where $\oplus$ denotes exclusive OR: it is one exactly when its two inputs differ. In this example x is a single ordinary bit and the ordered pair (y, z) is the witness. When $x = 0$, the OR gate requires $y = 1$, and the exclusive OR then requires $z = 0$. When $x = 1$, the OR gate already outputs one, and either pair with $y \neq z$ works. Thus

$$W_0 = \{(1, 0)\}, \quad W_1 = \{(0, 1), (1, 0)\}.$$

The accepting triples, written in the order xyz , are precisely 010, 101, and 110. Both fibers are nonempty, so $g(x) = \exists y, z f(x, y, z) = 1$ for both values of x . The checker uses three gates; the projected function is constant and needs no gates under the free-constants convention. The number of witness choices alone therefore cannot determine the difficulty of the projected function.

1.3 Existential choices are globally consistent

The witness is chosen once. Every occurrence of the same bit must use the same value. A minimal failure of independent local reasoning is

$$\begin{aligned} (\exists y \in \{0, 1\} \ y = 1) \wedge (\exists y \in \{0, 1\} \ y = 0) & \text{ is true,} \\ \exists y \in \{0, 1\} \ (y = 1 \wedge y = 0) & \text{ is false.} \end{aligned}$$

In the left expression the quantifiers have separate scopes; renaming their variables to y_1 and y_2 makes the independence explicit. In the right expression one value must satisfy both conditions. In

general, existence of a common witness implies existence of separate witnesses, but the converse fails. Independent local existence tests can admit combinations that no actual computation realizes.

The same issue is visible inside the running circuit. Hold $x = 0$ and $z = 1$. The relation $x \vee y = 1$ can be satisfied with $y = 1$, while $y \oplus z = 1$ can be satisfied with $y = 0$. Both local tests succeed, but no shared y makes the final AND equal one. A correct algorithm may split the two occurrences into temporary variables only if it also enforces their equality.

Forward evaluation of a circuit never encounters a directed cycle: each gate uses already determined predecessors. Yet the undirected pattern of dependencies can contain a cycle. In this example the value of y feeds both v_1 and v_2 , and those branches meet at the output gate. Ignoring directions gives the cycle y, v_1, f, v_2, y . Such a cycle expresses shared information, not a feedback loop in the computation. The later consistency construction keeps this sharing explicit through equality constraints. It is this graph of constraints, rather than the directed evaluation order alone, that controls how many unresolved choices an elimination algorithm must remember.

For the undirected graphs used below, a vertex's *degree* counts its incident edge ends. A graph is *cubic* when every degree is three and *subcubic* when every degree is at most three. A *simple* graph has no loops or parallel edges. Intermediate constraint graphs may have both: a loop joins a vertex to itself and contributes two to its degree, while parallel edges join the same pair of vertices more than once.

1.4 Existence and probability ask different questions

Giving a witness distribution changes the question. If Y is uniform on $\{0, 1\}^m$, then

$$\mathbb{P}[f(x, Y) = 1] = \frac{|W_x|}{2^m}, \quad g(x) = 1 \iff \mathbb{P}[f(x, Y) = 1] > 0.$$

A positive probability can be exponentially small. If a checker accepts only the witness 0^m , one uniform trial succeeds with probability 2^{-m} . For r independent trials the success probability is $1 - (1 - 2^{-m})^r \leq r2^{-m}$. Polynomially many such trials do not provide a constant success guarantee as m grows. This example concerns uniform guessing; the accepted witness is transparent, so it is not a lower bound against other search algorithms.

Randomized computation specifies how choices are distributed and requires an appropriate success or error guarantee. Existential acceptance specifies only that some legal choice succeeds. The two descriptions can use the same tree while placing different conditions on its leaves. A good randomized search algorithm may exploit the verifier's structure, but its guarantee does not follow merely from the existence of one accepting path.

1.5 From bounded certificates to polynomial verification

To discuss a complexity class, one verifier must handle inputs of every length. A *language* is a set of finite bit strings, representing its yes-instances. The class P consists of languages decided by a deterministic algorithm in time polynomial in the input length. A language belongs to NP when there are a polynomially bounded, polynomial-time computable witness length $p(n)$ and a deterministic polynomial-time verifier V such that

$$x \in L \iff \exists y \in \{0, 1\}^{p(|x|)} V(x, y) = 1. \quad (3)$$

One may take p to be an integer polynomial bound. Guessing the witness and checking it gives a polynomial-time nondeterministic machine. Conversely, recording the choices of a polynomial-time

nondeterministic machine gives a polynomial-length witness that can be checked by simulation. Illegal transition codes are rejected. This proves the equivalence of the path and verifier definitions at the level of acceptance. The question $P = NP$ asks whether every language with such efficient verification also has an efficient deterministic decision algorithm.

The fixed length in (3) is a convention that needs care when multiplicities matter. Suppose the original verifier accepts witnesses of any length at most $p(n)$. Encode such a witness by its length ℓ in $h = \lceil \log_2(p(n) + 1) \rceil$ bits, followed by $p(n)$ data bits. Require $0 \leq \ell \leq p(n)$ and require every data bit after position ℓ to be zero; run the original verifier on the first ℓ bits. Each original witness has exactly one valid encoding of length $h + p(n)$. Without the suffix-zero check, a length- ℓ witness would have $2^{p(n)-\ell}$ encodings. Existence would be preserved, but its contribution to a count would change.

Similarly, a record of machine choices is not automatically in bijection with accepting paths: unused choices after early halting or several bit codes for one transition can multiply representations. Canonical padding and a one-to-one transition encoding can remove these multiplicities. Whenever counting is claimed to be preserved, the relevant bijection must be supplied; an acceptance equivalence alone is insufficient.

For each fixed n , a polynomial-time verifier can be expressed by a polynomial-size Boolean circuit $f_n(x, y)$. One elementary construction encodes the verifier's configuration at each time step and connects a Boolean circuit for one transition to the next. A polynomial time bound limits both the number of steps and the tape region that can be visited, so this finite unrolling has polynomial size. The input and witness bits remain circuit inputs, while the machine description and time bound fix its wiring. This construction is effective across input lengths.

Encoding an accepting computation by jointly satisfied Boolean constraints is the classical foundation behind the SAT connection. Cook constructs a formula satisfiable exactly when a polynomial-time nondeterministic computation accepts, and also discusses bounded existential quantification over deterministic polynomial-time relations [8]. Levin's foundational formulation studies universal search problems [19]. The circuit presentation here uses the same verification principle. With x fixed, asking whether $f_n(x, y) = 1$ for some y is circuit satisfiability. Introducing internal gate values and enforcing each gate relation converts this to a finite constraint problem; once its input assignment is fixed, deterministic evaluation gives exactly one consistent assignment to those internal gate values.

SAT abbreviates Boolean satisfiability: does some assignment make the given formula or circuit output one? A *literal* is a variable or its negation, a *clause* is an OR of literals, and a formula in *conjunctive normal form* (CNF) is an AND of clauses. In a k -CNF each clause has at most k literals. Gate count and clause count are different size measures, so the SAT bounds below specify their circuit basis.

1.6 Four computational tasks and their outputs

For a fixed ordinary input, *decision* asks whether W_x is nonempty, *search* asks for one member when it is nonempty, and *exact witness counting* asks for the integer $|W_x|$. For a circuit with both ordinary and witness inputs left variable, two totals must also be separated:

$$N_{\text{pairs}} = \sum_{x \in \{0,1\}^n} |W_x| = \sum_{x,y} f(x, y), \quad N_{\text{proj}} = \sum_{x \in \{0,1\}^n} [W_x \neq \emptyset] = \sum_x g(x). \quad (4)$$

The brackets denote an indicator, equal to one when the condition holds and zero otherwise. The first total counts accepting input-witness pairs; the second is *projected model counting*, which counts

each successful ordinary input once. They agree when every nonempty fiber has size one, but not in general. In the running example they are three and two.

Across input lengths, the class $\#P$ consists of functions counting the accepting paths of polynomial-time nondeterministic machines. Equivalently, they count accepted fixed-length polynomially bounded certificates for a polynomial-time verifier, using the encoding discipline above. These are integer-valued functions, whereas NP describes decision problems. The distinction will matter when the appendix compares existence with exact counting.

An exact witness count decides existence by comparison with zero. It also supports search: fix successive witness bits to a branch with a nonzero completion count. The invariant is that the current prefix still extends to an accepted witness, so a full prefix obtained this way is a witness. This explains the logical reduction; preserving a particular algorithmic exponent across these restricted instances needs the additional analysis given later. A single decision answer, by contrast, provides no fiber size.

Counts also specify distributions. Uniformly choosing one of the three accepting triples in the example gives $\mathbb{P}[x = 1] = 2/3$, whereas uniformly choosing a successful projected input gives $\mathbb{P}[x = 1] = 1/2$. In general the marginal distribution of a uniform accepting pair assigns mass $|W_x|/N_{\text{pairs}}$ to x . Forgetting a sampled witness therefore favors larger fibers. The exact-counting algorithm in this paper counts satisfying assignments to all named circuit inputs; when these inputs are partitioned into x and y , it computes N_{pairs} , without automatically computing N_{proj} .

1.7 Circuit existence and uniform computation

A size bound for g asserts that a circuit computing g exists. A uniform SAT algorithm receives a circuit description and computes whether it has a satisfying assignment. These statements have different inputs and quantifiers. At each length there could be a small circuit without an efficient procedure for finding it. In particular, a projection that happens to be constant has a zero-gate circuit even if recognizing that fact from the supplied verifier is difficult.

Across lengths, P/poly denotes languages decided by polynomial-size circuit families, with no requirement that a single algorithm generate the family efficiently. The verifier construction above starts with a single polynomial-time algorithm and produces circuits effectively. An arbitrary family of polynomial-size verifier circuits need not have this uniformity, so its projected language should not be identified with NP merely from the size assertion. Likewise, the polynomial size of the verifier says little by itself about the minimum circuit size after projection; controlling that increase is a principal question here.

For an algorithm on an explicitly supplied circuit, the input includes its gate operations, wiring, output, and named inputs. Polynomial factors in the running-time bounds account for this description, including inputs that may become unused in normalization. For circuit-size bounds the gate count is instead the resource, with the basis and free-wiring conventions stated explicitly. Neither measure substitutes for the other.

The rest of the paper studies existential witnesses in this finite Boolean setting. Alternation adds universal as well as existential choices, and nondeterministic space measures memory along computations; both lead to further questions beyond the present scope. Our route is more specific: one accepting path gives one complete witness; fixing that witness makes verification deterministic; shared values require equality; and these equalities let us count consistent assignments by combining local tables. The next construction makes that route operational.

2 A complete example: from shared values to counting tables

The algorithm's basic question is what must remain known after part of a circuit has been processed. An accepting input determines every gate value, but a partial calculation must retain the values on which unprocessed gates still depend. We work through this distinction numerically before using graph width to bound the size of those partial calculations.

2.1 Gate equations preserve the objects being counted

Return to the circuit

$$v_1 = x \vee y, \quad v_2 = y \oplus z, \quad v_3 = v_1 \wedge v_2, \quad f(x, y, z) = v_3.$$

Here all three inputs are quantified when we count satisfying assignments. The symbols v_i denote gate values; the parameter b elsewhere still denotes the number of retained inputs. Introduce indicator tables

$$\begin{aligned} G_1(x, y, a) &= [a = x \vee y], & G_2(y, z, c) &= [c = y \oplus z], \\ G_3(a, c, d) &= [d = a \wedge c], & O(d) &= [d = 1]. \end{aligned}$$

Brackets equal one when their statement holds and zero otherwise. The letters a, c, d are possible values of v_1, v_2, v_3 , respectively. The exact count is

$$Z = \sum_{x, y, z, a, c, d \in \{0,1\}} G_1(x, y, a) G_2(y, z, c) G_3(a, c, d) O(d). \quad (5)$$

Why does this sum count inputs, although it also sums over gate values? For each fixed (x, y, z) , the first equation forces a , the second forces c , and the third then forces d . There is exactly one extension satisfying the three gate equations. The output factor keeps that extension exactly when the input accepts. Thus the extra variables introduce no multiplicity. This topological argument is the step needed for counting; preservation of mere satisfiability would not suffice.

The complete evaluation table makes this bijection visible:

x	y	z	v_1	v_2	v_3
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	0	0

The accepting triples are 010, 101, 110, so $Z = 3$. If x is ordinary and (y, z) are witnesses instead, the witness counts are 1 and 2, while both ordinary inputs have an accepting witness. Counting accepting pairs gives three; counting distinct positive projected inputs gives two.

2.2 Why a directed acyclic circuit has a consistency cycle

The incidence graph has a vertex for each of the six values x, y, z, v_1, v_2, v_3 , and a vertex for each of G_1, G_2, G_3, O . An edge records a value's occurrence in a table. Thus each gate table has three

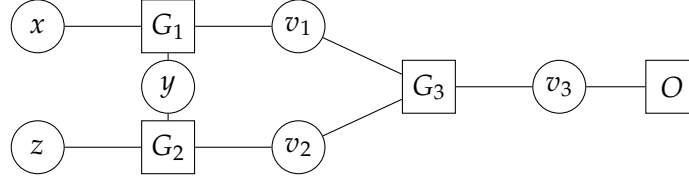


Figure 1: Circles enforce agreement between occurrences of one value; rectangles enforce gate equations or the output condition. The cycle runs through $y, G_1, v_1, G_3, v_2, G_2$.

incident edges and O has one: there are ten vertices and ten edges. The graph is connected. For a connected graph, its *cycle rank* $|E| - |V| + 1$ counts the edges left after a spanning tree is removed. Here the cycle rank is $10 - 10 + 1 = 1$.

There is no directed feedback in the original circuit. The undirected cycle records two uses of y whose branches later meet at G_3 . This is precisely the agreement a local calculation must preserve. In the all-quantified notation of Lemma 7.1, $q = b = 3$, $h = 0$, and $\ell = 6$, giving $\kappa = 6 - 3 - 3 + 1 = 1$ and $h + b + \kappa = q + 1 = 4$.

To obtain a network of small tables, give each incidence edge its own bit. At every circle of degree D , put the equality table

$$E_D(t_1, \dots, t_D) = [t_1 = \dots = t_D].$$

Here $E_1(0) = E_1(1) = 1$: a leaf input is free, not fixed to one. At each rectangle put its indicated gate or output table. Sum the product of all these tables over the ten edge bits. A nonzero product requires all copies of each value to agree. Each assignment satisfying the gate equations then determines exactly one assignment to all ten edges, and conversely. Consequently this edge-index sum is also Z .

An array with one coordinate per incident edge is called a tensor. Its contraction sums products over shared coordinates. This is the standard tensor-network perspective on circuit simulation [23]; the elementary zero-one correspondence above establishes the particular counting interpretation needed here.

2.3 Eliminate two inputs and inspect every surviving entry

First sum over x and z , retaining the common value y :

$$A(y, a) = \sum_x [a = x \vee y], \quad B(y, c) = \sum_z [c = y \oplus z].$$

Rows are indexed by $y = 0, 1$ and columns by the indicated gate value $0, 1$:

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (6)$$

For $y = 0$, exactly one x produces each possible OR output. For $y = 1$, neither x produces output zero and both produce output one; this explains the entry two. For each fixed y and XOR output c , exactly one z works, explaining all four ones in B .

Next sum over d . The output must be one, so

$$C(a, c) = \sum_d [d = a \wedge c][d = 1] = [a = c = 1].$$

The remaining sum collapses to

$$Z = \sum_{y,a,c} A(y,a)B(y,c)C(a,c) = \sum_y A(y,1)B(y,1) = 1 \cdot 1 + 2 \cdot 1 = 3. \quad (7)$$

The first term represents 101; the second represents 010 and 110. What would happen if the two uses of y were independently summed away? The result would instead be

$$\left(\sum_y A(y,1) \right) \left(\sum_{y'} B(y',1) \right) = (1+2)(1+1) = 6.$$

This counts assignments to a changed circuit with two independent inputs in place of the shared y . The extra three combinations disagree between the two branches. The equality table prevents precisely those combinations.

Pinning x exposes the two witness fibers without changing the network's edges. Replace its leaf table $(1,1)$ by $(1,0)$ for $x=0$ or $(0,1)$ for $x=1$. The resulting left tables are

$$A_{x=0} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad A_{x=1} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}.$$

Equation (7) now gives $0+1=1$ and $1+1=2$. Further pins give $Z_{x=0,y=0}=0$, $Z_{x=0,y=1}=1$, and $Z_{x=1,y=0}=Z_{x=1,y=1}=1$. A pin is evidence restricting an original input, not an additional independent choice.

2.4 The same operations answer existence and counting

For decision, replace every sum by OR and every product by AND. An intermediate entry now says whether at least one internal assignment works, rather than how many work. This is calculation in the Boolean semiring $(\{0,1\}, \vee, \wedge)$; counting uses the nonnegative-integer semiring $(\mathbb{N}, +, \cdot)$. Both support distributivity, so the same contraction identities apply. In particular, the entry two of A becomes one under Boolean contraction. Thresholding a nonnegative integer count at zero commutes with both operations:

$$[r+s > 0] = [r > 0] \vee [s > 0], \quad [rs > 0] = [r > 0] \wedge [s > 0].$$

For fixed ordinary x , the Boolean result is $g(x) = \bigvee_{y,z} f(x,y,z)$ and the integer result is $W(x) = \sum_{y,z} f(x,y,z)$. Hence $g(x) = [W(x) > 0]$. Counting projected inputs requires $\sum_x [W(x) > 0]$, which is not $\sum_x W(x)$. A sum-product calculation over all original inputs computes the latter. There is no automatic projected-counting guarantee from merely changing OR into addition.

An ordinary input can also remain symbolic throughout the calculation. For a separate small example, take

$$\psi(x_1, x_2; y) = (x_1 \wedge y) \vee (x_2 \wedge \neg y).$$

Fixing $y=0$ kills the first term and leaves x_2 ; fixing $y=1$ kills the second and leaves x_1 . The table indexed by y therefore contains the two circuit wires x_2, x_1 , rather than two evaluated bits. Boolean contraction of that index gives

$$\bigvee_{y \in \{0,1\}} \psi(x_1, x_2; y) = x_2 \vee x_1. \quad (8)$$

One OR gate connects these wires and works for all four ordinary inputs; constructing it does not require enumerating those inputs. By comparison, the integer witness count is $x_2 + x_1$, which is

two at $(x_1, x_2) = (1, 1)$ although the Boolean projection is one. In the general symbolic construction, each table entry similarly points to a circuit computing a Boolean function of the ordinary inputs. OR and AND combine those pointers by adding gates. When all original inputs are quantified for SAT, no ordinary-input wires remain symbolic, and the tables instead have constant Boolean or integer entries as in the running example.

Two small identities explain why multigraph details also matter. If two ports of one table are joined by a loop, both ports carry the same bit. For the two-port table

$$L = \begin{pmatrix} 2 & 3 \\ 5 & 7 \end{pmatrix},$$

closing the loop gives $L(0, 0) + L(1, 1) = 9$, not the sum 17 of all entries. For two tables joined by two parallel edges, take

$$U = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad V = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Joint contraction gives $\sum_{r,s} U(r, s)V(r, s) = 5 + 12 + 21 + 32 = 70$. Contracting just the r edge first gives

$$W(s, t) = \sum_r U(r, s)V(r, t) = \begin{pmatrix} 26 & 30 \\ 38 & 44 \end{pmatrix}.$$

The remaining edge becomes a loop, so its diagonal sum is again 70. Allowing its ends to vary independently would give 138. These are local integer-table fixtures, not additional gates of the running circuit.

2.5 What the frontier remembers in a larger instance

The example above reduces completely by the low-degree operations of Lemma 7.2; its reduced cubic graph is empty. Indeed, a nonempty connected cubic remainder would have cycle rank at least two, whereas the initial graph has rank one and reductions cannot increase it. The example demonstrates consistency and exact counting. It cannot test the asymptotic pathwidth theorem that handles nonempty cubic remainders.

For that next stage, order the vertices of a reduced graph. After processing a prefix, the *frontier* consists of the edges with one processed and one unprocessed endpoint. If there are w_i such edges, the table has 2^{w_i} entries. Each entry counts assignments to edges entirely inside the processed region, conditioned on the frontier bits. The unprocessed region has contributed no constraints yet. Thus a positive entry need not extend to a solution of the whole instance.

At the next vertex, write J for its edges into the processed region, R for its edges into the future, and S for frontier edges not incident to it. The old frontier is $S \cup J$ and the new one is $S \cup R$. For its local table T , the update is

$$P_{i+1}(\sigma, \rho) = \sum_{\eta \in \{0,1\}^J} P_i(\sigma, \eta) T(\eta, \rho). \quad (9)$$

Every extension of the new boundary has a unique choice of the consumed bits η and a unique assignment counted by the corresponding old entry. This proves the invariant by induction, starting with the empty prefix's scalar one. At the end the boundary is empty and its scalar is the desired count.

For a concrete transition with two consumed edges and one new edge, let $P_i(p, q)$ have rows $(1, 2)$ and $(3, 4)$, and let $T(p, q, r) = [r = p \oplus q]$. There are no spectator edges. The new table is $P_{i+1}(0) = 1 + 4 = 5$ and $P_{i+1}(1) = 2 + 3 = 5$. The old four entries are summarized by two entries

indexed by the value still needed in the future. This standalone transition illustrates a cubic vertex update without claiming that this particular table arose from our three-gate example.

The maximum frontier size of an order is its cutwidth. It controls table storage and arithmetic operation counts. It is not by itself a bit-time bound: integer entries also have a bit length. In the general construction there are $O(s + 1)$ edge indices, so an entry counts at most $2^{O(s+1)}$ internal assignments and has $O(s + 1)$ bits. Polynomial-cost exact integer arithmetic then converts the contraction bound to the bit bound proved in Corollary 7.8. The exponential table storage is allowed there. The accompanying exhaustive script checks the numbers, unique extensions, pins, and local contraction identities in this section; it implements neither the full SAT solver nor its layout theorem.

3 SAT and exact counting below four gates per input

The algorithmic entry point is a restricted-instance improvement over exhaustive search. Let C be a single-output Boolean circuit with u input bits and s internal gates. Work over the full binary basis B_2 : every Boolean function of at most two inputs is a unit-cost gate, constants and wiring are free, and fan-out is unrestricted.

Headline guarantee. For every fixed $\varepsilon > 0$, the construction counts the satisfying assignments of C , and therefore decides its satisfiability, in time

$$\boxed{\text{poly}_\varepsilon(s + u + 1) 2^{(1/4+\varepsilon)(s+1)}}. \quad (10)$$

The space bound allows the same exponential factor. Consequently, for each fixed $0 < \gamma < 4$, circuits with $s \leq (4 - \gamma)u$ admit an algorithm running in $\text{poly}_\gamma(u) 2^{(1-\eta)u}$ for some $\eta > 0$ depending only on γ . This beats the 2^u enumeration bound by an exponential factor. The restriction concerns the gate budget in B_2 , not the number of clauses in a CNF representation.

For example, take $\varepsilon = \gamma/[8(4 - \gamma)]$ in (10). Then

$$(1/4 + \varepsilon)(s + 1) \leq (1 - \gamma/8)u + (1/4 + \varepsilon),$$

so $\eta = \gamma/8$ is an explicit choice. The additive constant is absorbed into the prefactor. The positive slack is fixed before the input size grows; no exact endpoint exponent with a fixed polynomial prefactor is asserted.

Why the running time improves. Normalize the output cone without changing its function, leaving b used inputs. Each removed input contributes a factor of two to the final count. Encode the circuit's gate equations, consistent uses of every input and gate value, and the requirement that its output is one as a network of small tables. Every satisfying input assignment has exactly one extension to the internal values. The network's cycle rank κ satisfies

$$\kappa \leq s + 1 - b.$$

Split high-degree equality tables into trees, then contract low-degree parts, loops, and parallel edges with exact table identities. A nonempty residual simple cubic graph has at most $2(\kappa - 1)$ vertices. The constructive Fomin–Høie theorem [14], together with the width conversion proved below, gives a vertex order with at most $(1/3 + \delta)\kappa + O_\delta(1)$ edges crossing any frontier. Summing

over the frontier assignments evaluates the network using integer addition and multiplication. Intermediate counts have polynomial bit length.

Choosing between this contraction and enumeration of the b used inputs gives the sharper bound

$$\text{poly}_\delta(s + u + 1) 2^{\min\{b, (1/3+\delta)(s+1-b)\}}, \quad \delta > 0 \text{ fixed.} \quad (11)$$

Writing $\alpha = 1/3 + \delta$, the two exponents balance at $b = \alpha(s + 1)/(1 + \alpha)$, where their common value is $\alpha(s + 1)/(1 + \alpha)$. Its gate coefficient approaches $1/4$. Section 7 proves the graph construction, reductions, and frontier bound; Corollary 7.8 proves the uniform count-preserving algorithm and its bit complexity.

Status and scope. This is an asymptotic algorithm with exponential space, derived from established width and tensor methods with explicit circuit-specific accounting. The repository contains finite checks of supporting constructions, not an implementation or benchmark of the complete counting algorithm. Historical priority of the precise bound and its best-known unrestricted-space status remain unestablished; the statement-level comparison is in Section 15.

From SAT to understanding nondeterminism. The proof first asks what changes when only witness inputs are quantified and ordinary inputs remain symbolic. The same consistency budget then gives a nonuniform circuit-size exponent approaching $1/5$, alongside witness-coverage bounds, necessary conditions for strong amplification, and examples separating a complicated supplied circuit from an intrinsically hard function.

Returning to integer tables then gives the SAT bound and its consequences for search, indexing, and exact generation in Section 8.

4 The question and its quantifiers

We study

$$f : \{0, 1\}^{n+m} \longrightarrow \{0, 1\}, \quad g(x) = \bigvee_{y \in \{0, 1\}^m} f(x, y).$$

Circuit complexity measures the size of the *smallest* circuit for a function. It does not measure the cost of discovering that circuit. In particular, a circuit for g may exploit facts about f that are expensive to discover algorithmically.

Model. Unless explicitly stated otherwise, circuits are finite directed acyclic graphs over the full binary basis B_2 : every Boolean operation of at most two inputs is available at unit cost. Constants and input/output wiring are free; fan-out is unrestricted. The number of internal gates is the size. Write $C(h)$ for minimum size and

$$M_n = \max_{h: \{0, 1\}^n \rightarrow \{0, 1\}} C(h).$$

Changing to any fixed complete bounded-fan-in basis changes size by constant factors. Exact constants in gate-counting results, including the limiting coefficient $1/5$ in our size-dependent exponent, are basis-specific. All logarithms are base two, except $\ln$. Comparisons with $2^n/n$ assume $n \geq 2$.

Definition 4.1 (Worst-case projection complexity). For nonnegative integers n, m, s , let

$$D(n, m, s) = \max \{C(\exists y f(x, y)) : f : \{0, 1\}^{n+m} \rightarrow \{0, 1\}, C(f) \leq s\}.$$

Equivalently, if $N_m(h)$ is the minimum complexity of a verifier for h using at most m witness bits, then $D(n, m, s) = \max\{C(h) : N_m(h) \leq s\}$.

The budget formulation avoids requiring a verifier to have exactly some prescribed minimum complexity. Unused witness bits are allowed, so D is nondecreasing in m and s . Nevertheless, adding unused witnesses cannot create an intrinsic hardness amplification.

There is no lower bound of the form $C(g) \geq 2^m C(f)$ for *every* small verifier. Even requiring every input to be essential does not fix this:

$$f(x, y) = x_1 \oplus \cdots \oplus x_n \oplus y_1 \oplus \cdots \oplus y_m, \quad m \geq 1,$$

has $C(f) = n + m - 1$ over B_2 , while $g \equiv 1$ and $C(g) = 0$. The upper bound is an XOR chain; the lower bound follows from the input count in Lemma 6.1. A meaningful hardness assertion must therefore concern a worst case, a specified family, or a specified distribution.

Reading route and contributions. Sections 1–3 build the route from computation paths to shared witnesses, evaluate a complete counting-table example, and state the restricted SAT guarantee. The next two sections establish the elementary circuit-size bounds and explain why witness counts alone miss reusable computation. Section 7 improves the size exponent by combining a circuit-specific consistency budget with established graph-width tools, and proves the uniform counting algorithm in Section 7.6. Section 8 then derives polynomial-time counting at low cycle rank, sharper input-sensitive exponents, search, lexicographic indexing, and exact uniform generation. Section 9.1 derives necessary structure for amplification at the critical gate budget. Section 9.2 then realizes cubic kernels with nearly critical supplied budgets but easy projections, and calculates the exact gap between supplied and minimum verifier size. Sections 10.1 and 11 develop the joint sampling bound and analytical examples that separate it from the earlier estimates. The remaining sections distinguish typical truth tables, known lower bounds, conditional hardness, and open problems. Section 15 identifies which ingredients are classical and where priority of the combined statements is still unestablished; Appendix B records the research methods.

5 The universal ceiling and the first crossover

Proposition 5.1 (Exhaustive expansion). *If $C(f) \leq s$, then*

$$C(g) \leq 2^m s + (2^m - 1) = 2^m(s + 1) - 1.$$

Proof. For each witness $a \in \{0, 1\}^m$, fix the witness inputs of an s -gate circuit for f to a . The resulting circuit computes $f(x, a)$ using at most s gates. OR these outputs with $2^m - 1$ additional gates. $\square$

The classical Shannon–Lupanov theorem says

$$M_n = (1 + o(1)) \frac{2^n}{n} \quad \text{over } B_2. \quad (12)$$

See Lupanov’s synthesis paper and Wegener’s treatment [22, 35]. The following elementary argument establishes the order of growth needed here, without the sharp leading constant.

Proposition 5.2 (The order of the universal bound). $M_n = \Theta(2^n/n)$.

Proof. An ordered t -gate circuit can be specified by choosing an operation and two predecessors for each gate and one output wire. Padding unused gates if necessary, the number of functions of complexity at most t is at most

$$16^t(n + t + 2)^{2t+1}. \quad (13)$$

For a sufficiently small constant $c > 0$ and $t = \lfloor c2^n/n \rfloor$, the logarithm of this expression is strictly smaller than 2^n by a constant fraction. There are 2^{2^n} Boolean functions. This proves the lower bound, and in fact proves it for a $1 - o(1)$ fraction of functions.

For the upper bound, split the input into $u \in \{0, 1\}^{n-k}$ and $v \in \{0, 1\}^k$. Simultaneously compute every Boolean function of v . This library has size $O(2^{2^k})$: recursively, each function on j variables is a multiplexer between two already available functions on $j - 1$ variables, with constant gate cost. Summing over $j \leq k$ preserves the stated bound.

For each fixed $u = a$, the residual function $h(a, v)$ is one library output. A multiplexer tree controlled by u selects the appropriate output, using $O(2^{n-k})$ gates. For $n \geq 4$, choose $k = \lfloor \log n \rfloor - 1$, so $n/4 < 2^k \leq n/2$. The total is

$$O(2^{2^k} + 2^{n-k} + n) = O(2^{n/2} + 2^n/n) = O(2^n/n).$$

The finitely many smaller values of n are absorbed in the constant. $\square$

Corollary 5.3 (The basic envelope).

$$D(n, m, s) \leq \min\{2^m(s + 1) - 1, M_n\} = O(\min\{2^m(s + 1), 2^n/n\}). \quad (14)$$

For $s \geq 1$, the two estimates meet at the scale

$$s_* \asymp \frac{2^{n-m}}{n}, \quad \text{or, equivalently,} \quad m_* = n - \log n - \log s + O(1). \quad (15)$$

Thus if $s \gg 2^{n-m}/n$, universal synthesis is $o(2^m s)$. For an amplification factor 2^m to remain possible up to a constant factor, one needs $2^m s = O(2^n/n)$; allowing a specified loss factor relaxes this condition by that factor. The weaker condition $s \ll 2^n/n$ does not suffice.

Verifier size	Witness length at the crossover
$s = n^a$, fixed $a > 0$	$m = n - (a + 1) \log n + O(1)$
$s = 2^{\alpha n}$, fixed $0 < \alpha < 1$	$m = (1 - \alpha)n - \log n + O(1)$
$s = \Theta(2^n/n)$	$m = O(1)$, with constants determining the crossover

A ceiling is not a hardness theorem. Once $2^m s$ exceeds M_n , the upper bound flattens. This does *not* establish that $D(n, m, s)$ has reached M_n . Attainment is immediate only for $s \geq M_n$, when a hardest function of x itself is an allowed verifier. Whether much smaller verifiers can project to near-maximal functions is part of the question.

Verifiers near their own maximum. The relevant maximum for f is M_{n+m} , not M_n . If $C(f) \geq \alpha 2^{n+m}/(n + m)$ for a fixed $\alpha > 0$, then

$$\frac{C(g)}{2^m C(f)} = O\left(\frac{n + m}{\alpha n} 2^{-2m}\right). \quad (16)$$

For growing m this is an enormous relative saving. It says nothing by itself about whether g has small absolute complexity.

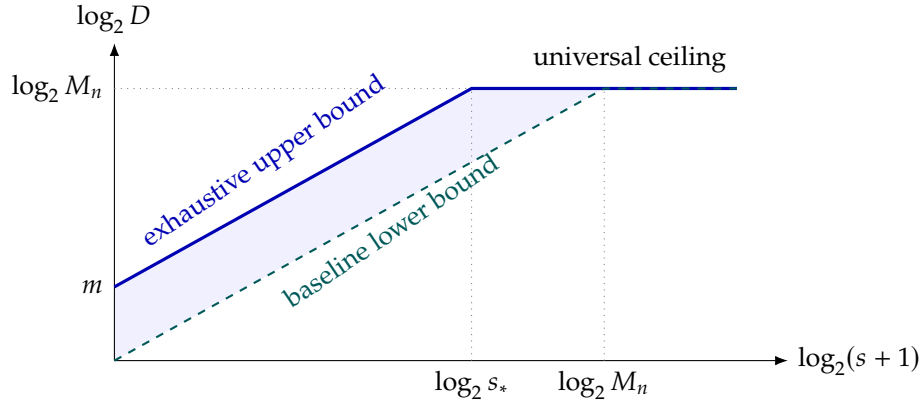


Figure 2: Schematic baseline bounds, suppressing additive constants on logarithmic axes. The shaded region represents a gap between bounds, not proved amplification. The lower bound is proved in Proposition 13.1. The structural bounds in Sections 6 and 7 can improve the upper curve.

6 Circuit structure yields a second easy range

Lemma 6.1 (Effective inputs). *The output cone of a circuit with $s \geq 1$ gates contains at most $s + 1$ distinct input vertices. In particular, only that many witness inputs can matter. A function depending essentially on t inputs requires at least $t - 1$ gates.*

Proof. Delete all vertices outside the output cone. If it contains t input vertices and s gates, its underlying undirected graph is connected, so it has at least $s + t - 1$ wires. Fan-in at most two gives at most $2s$ wires. Hence $t \leq s + 1$. Essential variables must occur in every circuit for the function. $\square$

For a chosen circuit, let a and b be its numbers of used ordinary and witness inputs. Constants can be propagated, removing irrelevant pins without increasing size in B_2 . Apart from constant functions, we can therefore take the output cone to have no constant vertices. The case of an output that is a single input is immediate below.

6.1 Sharing identical restrictions

For a gate v , let Y_v be the set of witness inputs in its syntactic dependency cone, and write $k_v = |Y_v|$. Let k be the corresponding number at the output.

Proposition 6.2 (A support-sensitive construction). *For a circuit with gates V and a gate as output,*

$$C(g) \leq \sum_{v \in V} 2^{k_v} + 2^k - 1. \quad (17)$$

Proof. For each v and each assignment $\alpha \in \{0, 1\}^{Y_v}$, create one copy of v . Its inputs are the copies of its predecessors indexed by the restrictions of α to their witness supports. Ordinary input wires are shared; witness inputs are constants. Induction along the circuit proves correctness. OR the 2^k output copies. $\square$

In particular, the part of the verifier independent of y need only be computed once. This bound charges syntactic supports; simplifying restricted gates can give further savings.

6.2 Cutting shared witness inputs and gate outputs

There is another construction that handles circuits close to trees. Its proof is useful because it pinpoints the consistency that must be retained when branches share witnesses. The underlying expansion of copying constraints into consistent zero/one choices is established in tensor-network counting [5, Theorem 17 of the author version]. Here we keep ordinary inputs symbolic and track the B_2 gate budget.

Lemma 6.3 (Eliminating witnesses in a forest). *Suppose a collection of formulas has q gates in total, ordinary input variables may repeat, and each witness variable occurs at most once in the entire collection. The assertion that there is a witness assignment making every formula output its prescribed bit has a deterministic circuit of size $O(q + \ell + 1)$, where ℓ is the number of leaves and roots.*

Proof. For each formula node v , compute two predicates $P_v^0(x), P_v^1(x)$: which output values can its subtree realize by assigning its witnesses? At an ordinary input x_i , the pair is $(\neg x_i, x_i)$; at a witness leaf it is $(1, 1)$; and at a constant c it is $(1 - c, c)$. For a binary gate $v = h(u, w)$, use

$$P_v^c = \bigvee_{\substack{d,e \in \{0,1\} \\ h(d,e)=c}} (P_u^d \wedge P_w^e). \quad (18)$$

For a unary gate, use the analogous disjunction over its possible input values. The two subtrees have disjoint witness sets, so the indicated choices can be realized simultaneously for each fixed x . Each gate costs only a constant number of gates for its two predicates. Take the AND of the appropriate predicates at the roots. Witness sets of different trees are also disjoint. Repeated ordinary inputs cause no problem: they keep the same fixed values throughout. $\square$

Theorem 6.4 (Simulation by fixing shared vertices). *Consider a constant-free output cone with $s \geq 1$ gates, a ordinary inputs, and b witness inputs. Let r count the witness-input and gate vertices whose fan-out is at least two. Then*

$$C(g) = O((s + 1)2^r), \quad r \leq s + 1 - a - b. \quad (19)$$

Ordinary input vertices are not included in r .

Proof. Let S be the set of the r indicated vertices. Enumerate a bit assignment $\beta \in \{0, 1\}^S$. Replace every use of a vertex in S by its assigned bit. If that vertex is a gate, retain its defining gate and impose the equation that its computed output equals its assigned bit. Also require the original circuit output to be one.

After these cuts, every remaining gate output is used at most once, and every unfixed witness input is used at most once. The defining equations for the cut gates, together with the original output condition, form a forest of formulas. It has $O(s + 1)$ total gates, leaves, and roots; ordinary inputs may repeat. Lemma 6.3 gives an $O(s + 1)$ -gate circuit for this branch. OR the 2^r branches.

For correctness, a satisfying assignment of the original circuit supplies a branch assignment by taking the actual values of the cut vertices. Conversely, a satisfying branch supplies compatible assignments to the remaining, disjoint witness sets. Restoring the cut wires, the retained defining equations ensure by topological induction that every assigned gate value equals its original computed value. Thus the original output is one. The defining equations are essential; guessing gate values without checking them would be unsound.

For the bound on r , let E be the number of wires. Every vertex except the output has fan-out at least one. Therefore

$$\sum_{v \neq \text{out}} (\text{fanout}(v) - 1) = E - (s + a + b - 1) \leq s + 1 - a - b.$$

Every counted vertex contributes at least one to this nonnegative sum. $\square$

Corollary 6.5 (A size-dependent universal bound). *Every verifier of complexity s satisfies*

$$C(g) = O\left((s+1)2^{\min\{m, n, (s+1)/3\}}\right). \quad (20)$$

Consequently this estimate can be added to the minimum in (14).

Proof. Normalize an optimal verifier circuit as above and let $s' \leq s$ be its remaining size. For a nontrivial output cone put $d = s' + 1 - a - b \geq 0$. Exhaustive expansion costs $O((s' + 1)2^b)$. Truth-table synthesis on the a used ordinary inputs costs $O((s' + 1)2^a)$, even by a minterm construction since $a \leq s' + 1$. Theorem 6.4 costs $O((s' + 1)2^d)$. Taking the minimum gives

$$C(g) = O((s' + 1)2^{\min\{a, b, d\}}).$$

Since $a + b + d = s' + 1$, this exponent is at most $(s' + 1)/3$; it is also at most n and m . Constant and single-input outputs obey the bound directly. $\square$

For example, if a verifier depends on all $n + m$ inputs and has $s = n + m - 1 + d$ gates, then $C(g) = O((s + 1)2^d)$. If $d = O(\log s)$, the projection has polynomial-size circuits even when m is large. Without a full-input-dependence assumption, the unconditional $2^{(s+1)/3}$ bound still saves an exponential factor over 2^m whenever $s + 1 \leq (3 - \delta)m$ for fixed $\delta > 0$.

Remark 6.6 (What the coefficient means). The factor $1/3$ follows from balancing three quantities: used ordinary inputs, used witness inputs, and the available excess fan-out. It is an elementary upper bound, not an optimality claim or a known exact transition. The argument is specific to gate count and fan-in two.

7 A stronger simulation from the consistency graph

Fixing every shared vertex is sufficient, but it discards the arrangement of those vertices. The graph may permit a sequence of small intermediate tables even when the total number of shared values is large. We now use that arrangement. The proof combines a circuit-specific count of consistency constraints with a published theorem on pathwidth. Its quantitative consequence improves the preceding $1/3$ exponential rate to $1/5$.

The basic operation is simple: keep a table of which assignments to the current boundary can be extended through the processed part. Each table entry is a circuit on x . Ordinary inputs remain shared circuit wires; they are never enumerated as table indices.

7.1 Separate ordinary computation from consistency

Normalize and prune the supplied circuit. If its output has no syntactic witness dependency, its existing circuit computes the projection. If its output is a witness input, the projection is one. Otherwise partition the gates into p gates independent of the witnesses and $q \geq 1$ dependent gates. Compute the independent gates once. Let $h_1(x), \dots, h_h(x)$ be the distinct independent signals entering dependent gates, including direct ordinary inputs, and let b be the number of used witnesses. Let ℓ count input pins of dependent gates supplied by a witness or a dependent gate. Then

$$\ell + h \leq 2q. \quad (21)$$

Here h counts distinct signals, while ℓ counts pins, with multiplicity. Every dependent gate has at least one dependent input, so it has at most one independent input.

Introduce a consistency variable for every witness and every dependent gate output. A gate v contributes the constraint

$$A_v = [z_v = \text{op}_v(\text{predecessor values})],$$

and the output contributes $[z_{\text{out}} = 1]$. The ordinary signals in these expressions remain functions of x . Form a bipartite incidence graph H with one vertex for each consistency variable and each constraint, and one edge for each incidence. Retain multiple incidences if a value supplies two pins.

Lemma 7.1 (The consistency budget). *The graph H is connected and has*

$$|V(H)| = 2q + b + 1, \quad |E(H)| = q + \ell + 1.$$

Its cycle rank $\kappa = |E(H)| - |V(H)| + 1$ therefore satisfies

$$\boxed{\kappa = \ell - q - b + 1, \quad h + b + \kappa \leq q + 1.} \quad (22)$$

If the full cone has $s = p + q$ gates and a used ordinary inputs, then also $\kappa \leq s + 1 - a - b$.

Proof. Every dependent gate and used witness has a directed path through dependent gates to the output. The incidence graph subdivides those connections and adds the output constraint, proving connectivity and the counts. Equation (21) gives the boxed inequality. For the last assertion, delete the ordinary inputs and independent gates from the full cone's underlying undirected multigraph. The remaining graph has $q + b$ vertices and ℓ edges, and hence cycle rank κ . Deletion cannot increase cycle rank, as follows by restricting the binary cycle space. The full cone has at most $2s$ edges and $s + a + b$ vertices, giving the claimed bound. $\square$

The conjunction of the constraints, quantified over all consistency variables, equals g . An original witness assignment extends uniquely to its gate values. Conversely, the equations force those values by topological induction. This observation preserves the consistency that naive separate existential evaluation loses.

7.2 Small tables reduce the graph to a cubic graph

Give every incidence edge its own binary index. At a variable vertex, place the equality table: its entry is one exactly when all incident indices agree. At a constraint vertex, place the constraint's truth table on its incident indices. A degree-one equality table has both entries one. A gate table has at most three indices. After fixing its indices, its entry is a constant, some $h_i(x)$, or its complement. Thus the p independent gates and at most h additional complements prepare every table entry.

We call these arrays *Boolean tensors*; no numerical approximation is involved. Contracting an edge means ORing over its binary value and ANDing the two incident entries. Distributivity proves that contraction preserves the represented function, pointwise for each fixed x .

Lemma 7.2 (A cubic reduction with linear circuit cost). *The consistency tables reduce, using $O(q+1)$ extra gates, to a scalar factor and either an empty graph or a simple connected cubic graph K . If $N = |V(K)| > 0$, then*

$$N = 2(\kappa(K) - 1) \leq 2(\kappa - 1).$$

Every surviving table still has at most eight entries, each represented by an already constructed circuit wire.

Proof. First replace each equality vertex of degree $D \geq 4$ by a tree of $D - 2$ ternary equality vertices. The internal edge values are uniquely determined when the external values agree. This adds $D - 3$ vertices and edges, preserving cycle rank and connectivity. The new equality vertex count is at most the sum $q + \ell + 1$ of the old variable degrees. Including constraint vertices, the resulting subcubic graph has

$$N_0 \leq q + 1 + (q + \ell + 1) \leq 4q + 2$$

vertices. A loop counts twice toward degree.

Apply the following exact table operations until none applies.

1. Remove a rank-zero table and AND its scalar into an accumulator.
2. For a loop, identify its two ports and OR the two diagonal entries. There are at most two remaining entries. Summing all four combinations would violate the equality of the loop's two ends.
3. At a loop-free vertex of degree one or two, contract one edge into its neighbor. The resulting rank is at most three. Each of at most eight entries needs two ANDs and one OR, costing at most 24 gates. Any second edge between the same vertices becomes a loop.
4. If two cubic vertices share two edges, contract both at once. The four resulting entries each use four ANDs and three ORs, costing 28 gates. Three parallel edges instead produce one scalar using at most 15 gates.

Each operation decreases $|V| + |E|$, costs at most 32 gates, and keeps degree at most three. Initially $|E| \leq 3N_0/2$, so $80N_0$ gates suffice for all reductions, including the scalar accumulator.

Contracting an ordinary edge preserves cycle rank; removing a loop lowers it by one; contracting j parallel edges lowers it by $j - 1$. The remaining nonempty graph stays connected. At termination it is simple and every degree is three. Hence $|E(K)| = 3N/2$, and its cycle rank is $N/2 + 1$. This proves the assertion. $\square$

The treatment of loops and parallel edges matters. Silently deleting them as graph-theoretic nuisances would change the represented function. The reductions above manipulate their actual tables at a charged cost.

7.3 From a path decomposition to a circuit

A *path decomposition* is a sequence of bags containing all vertices, containing both ends of every edge in some bag, and containing each vertex in a consecutive interval of bags. Its width is the largest bag size minus one; pathwidth is the minimum such width. The cutwidth of a vertex ordering is the largest number of edges crossing between a prefix and the remaining vertices.

Lemma 7.3 (A width conversion for cubic graphs). *A simple cubic graph of pathwidth k has a vertex ordering of cutwidth at most $k + 2$.*

Proof. Given bags $B_1, \dots, B_L$, represent v by $I_v = [\text{first}(v) - 1/4, \text{last}(v) + 1/4]$. At most $k + 1$ intervals contain any point. For each edge $e = uv$ choose a distinct timestamp t_e in the interior of $I_u \cap I_v$, which is nonempty. Place v at the median t_v of its three incident timestamps and order the vertices by these medians.

Consider a cut at a point t distinct from all timestamps. For a crossing edge with $t_u < t < t_v$, charge it to u if $t_e > t$, and to v otherwise. The charged interval contains t . A vertex receives at

most one charge: it has just one incident timestamp strictly above its median and just one strictly below it. Thus there are at most $k + 1$ crossing edges.

At a cut between tied medians, at most two vertices are tied, because the edge timestamps are distinct and an edge has two endpoints. Apply the same charging rule to every other crossing edge. Only the edge whose timestamp equals the common median can remain uncharged, adding at most one. This proves the $k + 2$ bound and gives an explicit ordering. $\square$

This width conversion is consistent with established graph-search relations; it is included with a proof to make the numerical exponent checkable. Section 15 records the attribution.

Lemma 7.4 (Symbolic frontier contraction). *Given an ordering of K of cutwidth w , its tensor value has a circuit using fewer than $4N2^w$ additional gates.*

Proof. Let F_i be the edges leaving the first i vertices. For each $\alpha \in \{0, 1\}^{F_i}$ maintain a wire $P_i(\alpha; x)$: the OR over internal edge assignments of the AND of all processed table entries, with the boundary fixed to α . Start with $P_0 = 1$.

At the next vertex let J be its j edges into the processed prefix. For each new boundary assignment, OR over the 2^j assignments to J the AND of the appropriate old entry and local table entry. This preserves the stated meaning by distributivity. Its gate cost is

$$2^{|F_i|}(2^{j+1} - 1) < 2^{|F_i|+j+1}.$$

Because $|F_i| = |F_{i-1}| + 3 - 2j$ and both frontier sizes are at most w , one has $|F_i| + j \leq w + 1$: use the new frontier for $j = 0, 1$ and the old frontier for $j = 2, 3$. Each vertex therefore costs less than 2^{w+2} gates. At the last vertex there are no boundary indices and the single entry is the desired scalar. $\square$

The complete finite construction costs at most

$$p + h + 80N_0 + 4N2^w + 1 \leq p + 400(q + 1)2^w. \quad (23)$$

For an empty reduced graph the frontier construction is unnecessary; take $N = w = 0$ and use the scalar accumulator. Ordinary computation is paid once, including when its result appears in many boundary entries.

7.4 The improved exponential rate

We use the following published graph theorem: for every $\eta > 0$ there is n_η such that every simple subcubic graph on $N > n_\eta$ vertices has pathwidth at most $(1/6 + \eta)N$. A suitable decomposition can be constructed in polynomial time for fixed η [14, Theorem 5 and Section 4].

Theorem 7.5 (Projection controlled by consistency cycles). *For every $\delta > 0$ there is a constant A_δ such that*

$$C(g) \leq p + A_\delta(q + 1)2^{(1/3+\delta)\kappa}. \quad (24)$$

For example, $A_\delta = 400 \cdot 2^{n_{\delta/2}+2}$ suffices using the threshold in the cited theorem. No uniform small value of this constant, or endpoint bound $O((q + 1)2^{\kappa/3})$, is asserted.

Proof. Use the cubic reduction and the graph theorem with $\eta = \delta/2$. Since $N \leq 2\kappa$, Lemma 7.3 gives $w \leq (1/3 + \delta)\kappa + 2$ for large N . The finitely many smaller graphs are covered by adding $n_{\delta/2}$ to the exponent. Apply (23). $\square$

Theorem 7.6 (A universal 1/5 exponential rate). Set $\alpha = 1/3 + \delta$ and $\beta = \alpha/(1 + 2\alpha)$. With the preceding notation,

$$C(g) \leq p + O_\delta((q+1)2^{\min\{h, b, \alpha\kappa\}}) \leq p + O_\delta((q+1)2^{\beta(q+1)}). \quad (25)$$

In particular, for every $\varepsilon > 0$,

$$D(n, m, s) = O_\varepsilon\left(\min\left\{\frac{2^n}{n}, (s+1)2^m, (s+1)2^{(1/5+\varepsilon)(s+1)}\right\}\right). \quad (26)$$

The comparison with $2^n/n$ assumes $n \geq 2$ as before.

Proof. Two alternatives to (24) are $p + O((q+1)2^b)$ by witness enumeration and $p + M_h$ by treating the h independent signals as formal input bits. Substituting their actual values after synthesis is valid even when those values are correlated. The bound $M_h = O(2^h)$, also valid for $h = 0, 1$, suffices. Take the minimum of these three estimates. If $z = \min\{h, b, \alpha\kappa\}$, the consistency budget gives

$$(2 + 1/\alpha)z \leq h + b + \kappa \leq q + 1.$$

This proves (25). Since $\beta \rightarrow 1/5$ as $\delta \downarrow 0$, and $p + q \leq s$ after normalization, the claimed universal bound follows. Trivial outputs were handled before the construction. $\square$

This is a circuit-size theorem. In particular, the synthesis of an arbitrary function on the formal inputs does not promise that its truth table can be discovered in the size of the resulting circuit. The numerical rate comes from three precise facts: $h + b + \kappa \leq q + 1$, at most 2κ vertices in the cubic reduction, and pathwidth at most $N/6 + o(N)$. It is not a claim that the coefficient 1/5 is optimal.

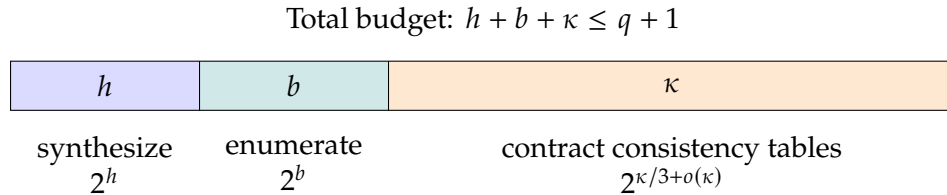


Figure 3: The limiting balance is $h : b : \kappa = 1 : 1 : 3$. All three exponents then approach $(q+1)/5$. This balances upper bounds; it supplies neither a hard example nor a proof that this rate is optimal. Ordinary preprocessing costs p additional gates, once.

Why the independent part should stay outside the exponent. For $t \geq 3$, put $a_i = x_{2i-1} \wedge x_{2i}$ and

$$H_t(x) = \bigoplus_{1 \leq i < j \leq t} (a_i \wedge a_j), \quad f_t(x, y) = H_t(x) \wedge (y_1 \vee \cdots \vee y_t).$$

Compute all a_i , their pair products, an XOR chain, the witness OR chain, and the final AND. This supplied circuit has $p = t^2 - 1$, $q = b = t$, $h = 1$, and $\kappa = 0$. The projection is H_t , with a circuit of size p . Every ordinary input is essential: set one other a_j to one, all remaining pairs to zero, and the other bit of the chosen pair to one. Every witness is essential when $H_t = 1$ and the other witnesses are zero. Thus unused inputs do not explain the simplification.

The full undirected circuit graph contains a subdivision of K_t , with the a_i as branch vertices and their pair-product gates on the edges. Its treewidth is at least $t - 1$, although the consistency graph is a tree. Also $r = t$ for the old shared-vertex bound, whereas the new structural bound is $O(t^2)$. Coverage independently detects a one-witness cover in this example. The example explains a distinction between structural parameters, not a circuit lower bound or an example attaining the $1/5$ exponent.

7.5 Condition a few values before eliminating the rest

Corollary 7.7 (Conditioning and componentwise elimination). *Fix t consistency variables in every possible way, retaining all defining equations. Let κ_* be the largest cycle rank of a connected component of the remaining incidence graph. Then*

$$C(g) \leq p + O_\delta \left((q + 1) 2^{t + (1/3 + \delta)\kappa_*} \right).$$

Proof. In each branch, distinct components have disjoint quantified variables. Their projections may therefore be ANDed for each fixed x . Apply the construction to each component; their total size is $O(q + 1)$ and their individual exponents are bounded by $(1/3 + \delta)\kappa_*$. OR the 2^t branches. The independent signals remain shared. Consistency equations make the conditioning sound, exactly as in Theorem 6.4. $\square$

Conditioning combined with elimination is an established algorithmic architecture. Here it supplies a refinement of the explicit circuit bound, useful when a small set of shared values separates otherwise difficult parts. The relevant exponent uses the largest residual component, rather than the sum of its components' cycle ranks.

7.6 The uniform SAT and exact-counting algorithm

We now prove the algorithmic guarantee highlighted in Section 3. The nonuniform theorem used synthesis on formal ordinary inputs. The all-quantified case has no such step. Its explicit table operations also preserve counts, which gives the following algorithmic consequence. This statement includes its proof; the unresolved issue in Section 15 is comparison and priority.

Corollary 7.8 (Counting by the gate-minus-input budget). *Fix $\delta > 0$ and write $\alpha = 1/3 + \delta$. Given a single-output B_2 circuit with s gates and u input bits, one can count its satisfying inputs in time*

$$\text{poly}_\delta(s + u + 1) 2^{\min\{b, \alpha(s+1-b)\}},$$

where b is the number of inputs remaining in a normalized output cone. In particular, for every fixed $\varepsilon > 0$, the time is

$$\text{poly}_\varepsilon(s + u + 1) 2^{(1/4 + \varepsilon)(s+1)}.$$

The space used is at most polynomial times the same exponential factor. A polynomial-space bound is not asserted.

Proof. Prune the output cone and propagate constants and unary redundancies without changing the function. Inputs removed from the cone are free; their contribution is a final multiplicative factor 2^{u-b} . Constant and designated-input outputs are evaluated directly. For the remaining circuit quantify all b inputs, so there are no independent signals. Its consistency rank satisfies $\kappa \leq s + 1 - b$.

Replace Boolean OR and AND in the tensor construction by integer addition and multiplication. The original tables contain only zeros and ones. For each satisfying input, the circuit equations

specify exactly one gate-value assignment. Incidence equality tables specify exactly one set of equal edge values, and their ternary expansions have exactly one internal extension. Thus the initial tensor contraction counts satisfying inputs, with no spurious multiplicity.

Every reduction in Lemma 7.2 is a distributive identity, including diagonal loop sums and joint parallel-edge sums. It remains valid over the nonnegative integers. More explicitly, a reduced table entry is the number of internal edge assignments in its contracted region that satisfy the original tables, conditional on its boundary. A contraction combines disjoint regions and sums over their shared indices, preserving this interpretation. The same invariant holds for the frontier entries in Lemma 7.4.

There are $O(s + 1)$ indices after equality expansion. Each table entry is therefore bounded by $2^{O(s+1)}$, so its bit length is $O(s + 1)$. Integer additions and multiplications have polynomial bit cost; the last factor 2^{u-b} only shifts the answer. For fixed δ , the cited constructive pathwidth theorem and median conversion find the required order in polynomial time. The bounded-size graphs below its threshold are handled by exhaustive order search, a constant depending on δ . The operation count is consequently polynomial times $2^{\alpha(s+1-b)}$, with exponential-size frontiers allowed.

Alternatively enumerate the 2^b used-input assignments, evaluate the circuit, and count the accepting ones. Choose the cheaper proved bound. Finally,

$$\min\{b, \alpha(s + 1 - b)\} \leq \frac{\alpha}{1 + \alpha}(s + 1), \quad \frac{\alpha}{1 + \alpha} \rightarrow \frac{1}{4}.$$

This proves both time estimates and the stated space majorant. $\square$

Testing whether the count is nonzero decides circuit satisfiability. For any fixed $0 < \gamma < 4$, the bound also gives a running time $2^{(1-\Omega_\gamma(1))u} \text{poly}(u)$ when $s \leq (4 - \gamma)u$, after choosing a sufficiently small fixed ε . This consequence uses exponential space and the exact B_2 gate convention. Its numerical comparison with previously reported circuit-SAT algorithms deserves particular scrutiny.

8 What the algorithm gives beyond a yes-or-no answer

The consistency construction yields more than a gate-only exponent. It identifies which part of a supplied circuit makes counting expensive, and its exact counts can guide the construction of individual solutions. These are consequences of the proved algorithm, using established conditioning and self-reduction ideas. They are not claims of historical priority for search or sampling from counts.

8.1 The excess above a tree is a useful parameter

Let s be the supplied gate budget, u the declared input count, and b the inputs left in a nontrivial normalized output cone. Define

$$d = s + 1 - b.$$

The input-count lemma ensures $d \geq 0$. The graph proof gives $\kappa \leq d$: every used input consumes one unit of the same budget that would otherwise permit a consistency cycle. This is a statement about the supplied representation, not its minimum possible size.

Corollary 8.1 (Counting with logarithmic excess). *For each fixed $\delta > 0$, exact counting takes time*

$$\text{poly}_\delta(s + u + 1) 2^{(1/3+\delta)d}.$$

In particular, for every fixed $c \geq 0$, circuits with $d \leq c \log(s + u + 1)$ can be counted in polynomial time. The same conclusion holds whenever $\kappa = O(\log(s + u + 1))$, even if the gate-minus-input estimate d is larger.

Proof. The proof of Corollary 7.8 first obtains time $\text{poly}_\delta(s + u + 1)2^{(1/3+\delta)\kappa}$ and then bounds κ by d . If $d \leq c \log(s + u + 1)$, the exponential factor is at most $(s + u + 1)^{c(1/3+\delta)}$. Fixing, for example, $\delta = 1/6$ makes the total a polynomial of fixed degree. The same calculation applies directly to κ . $\square$

A binary AND tree on u inputs has $s = u - 1$, $b = u$, and $d = 0$. Its single accepting assignment is easy to count even though there are 2^u candidate assignments. The explanation is not that few solutions make counting easy: it is that the circuit has no excess consistency to maintain. The running three-gate example has $b = 3$ and $d = 1$; its entire network also reduces by the elementary table operations. Neither example needs the asymptotic graph theorem.

A sharper reading of the four-gates threshold. The $1/4$ coefficient protects against every possible value of b . If all declared inputs survive normalization, $b = u$, the more precise bound has limiting input coefficient $(c - 1)/3$ at gate budget $s \leq cu$, provided $1 \leq c < 4$. In contrast, the gate-only coefficient is $c/4$. For example:

Gate budget	Arbitrary used-input count	All u inputs remain used
$s \leq 2u$	$1/2$	$1/3$
$s \leq 3u$	$3/4$	$2/3$
$s \leq (7/2)u$	$7/8$	$5/6$

The entries are limiting coefficients in an exponent of 2^u . Every entry requires arbitrarily small fixed positive slack and a polynomial prefactor depending on that slack; these are not timing measurements or endpoint bounds. Substituting $b = u$ into the proved formula gives the right column directly.

More generally, with $r = b/u$ and $c = s/u$, the limiting envelope is

$$\min\{r, (c - r)/3\},$$

up to fixed positive slack and the additive gate-budget constant. Increasing b makes enumeration more expensive but leaves fewer cycles for contraction. The two costs balance near $b = (s + 1)/4$. Thus the most expensive value in this guarantee need not be the largest number of used inputs. This is a balance of upper bounds, not an example attaining either running-time bound or a hardness theorem. Unused inputs merely multiply the exact answer by a power of two; writing that answer costs polynomially many bits.

8.2 Conditioning without losing the original budget

Suppose some input values have already been chosen. We want to count how many accepting assignments extend those choices. Re-normalizing the restricted circuit changes b , so substituting its new parameters into $s + 1 - b$ does not itself prove preservation of the original bound. There is a simpler way: keep the original graph and change its tables.

Preserving only the unconditioned total would not justify this step. The predicates $x \wedge \neg z$ and $\neg x \wedge z$ each have one model on the named universe (x, z) , but pinning $x = 1$ leaves one model of the first and none of the second. We need the stronger invariant that every boundary-conditioned count is preserved.

Lemma 8.2 (Pinning preserves the counting layout). *Fix a nontrivial normalized output cone and its original consistency graph of rank κ . For any partial assignment e to the u declared inputs, the number $Z(e)$ of satisfying assignments extending e can be computed in time*

$$T_\delta = \text{poly}_\delta(s + u + 1) 2^{\min\{b, (1/3+\delta)\kappa\}}, \quad \delta > 0 \text{ fixed}, \quad (27)$$

using the original graph reduction and vertex layout for every query. Space is bounded by a polynomial times the same exponential factor. Constant and designated-input outputs can be handled directly.

Proof. At a used input fixed by e , replace its equality table by the table that is one precisely when all incident values equal the prescribed bit. No vertex, edge, or index is added. When a high-degree equality table is expanded as a ternary tree, impose the prescribed bit at one of its ternary tables. The other equality tables propagate that bit through the tree. A consistent external assignment still has exactly one internal extension, and an inconsistent one has none.

The low-degree, loop, and parallel-edge reductions depend only on the graph and are exact integer identities for the modified tables as well. The same residual graph and layout therefore give the same frontier sizes and operation bound. All entries remain counts of assignments to $O(s + 1)$ internal indices, with polynomial bit length. For each originally unused input not fixed by e , multiply the answer by two; pinned unused inputs contribute one instead. This factor has at most $u + 1$ bits.

Alternatively enumerate the original b used input bits, discarding assignments inconsistent with e . This costs at most a polynomial times 2^b . Choose between the two original guarantees. The graph and layout can be prepared once, while the integer tables are recomputed for each query. Storing queries sequentially suffices for the stated space bound. $\square$

8.3 Search, rank, and exact uniform generation

Order the declared input bits once and use lexicographic order on their assignments, with $0 < 1$. A prefix p has two disjoint sets of extensions, so its exact counts satisfy

$$Z(p) = Z(p0) + Z(p1).$$

This identity is the bridge from counting all solutions to selecting one. Count-guided construction and generation are classical; see Jerrum et al. [16, Theorem 3.3 and the discussion of fair coins in Section 2]. The point here is that pinning retains the original circuit's numerical bound.

Corollary 8.3 (Constructing and indexing satisfying assignments). *Within a polynomial factor of (27), one can deterministically find a satisfying assignment or report that none exists; compute the zero-based lexicographic rank of a supplied satisfying assignment; or reconstruct the satisfying assignment of any supplied rank R with $0 \leq R < Z(\emptyset)$. The same gate-only coefficient approaching $1/4$ consequently applies to all three tasks.*

Proof. First compute $Z = Z(\emptyset)$. If $Z = 0$, report unsatisfiability. Otherwise, maintain a prefix p , an integer R , and the invariant $0 \leq R < Z(p)$. Compute $A = Z(p0)$. If $R < A$, append zero; otherwise append one and replace R by $R - A$. The identity above preserves the invariant. After u steps the prefix is the unique satisfying assignment of the original rank. Taking $R = 0$ gives the first satisfying assignment.

Conversely, verify a supplied assignment by direct circuit evaluation. Along its prefixes, add $Z(p0)$ to an accumulator whenever its next bit is one. These are exactly the disjoint lexicographically earlier branches, so their sum is the rank. Both procedures use at most $u + 1$ count queries and arithmetic on $O(u + 1)$ -bit integers. Sequential queries multiply time by a polynomial and do not multiply space. Finally use $\kappa \leq s + 1 - b$ and the same balancing calculation as in Corollary 7.8. $\square$

For the running example the accepting assignments are 010, 101, 110, in that order. The zero branch for the first bit has one accepting extension and the one branch has two. Unranking $R = 2$ therefore selects the one branch and changes the residual rank to one; continuing in the same way selects 110. Choosing each nonempty branch with probability one half would give the first assignment probability $1/2$, rather than $1/3$.

Corollary 8.4 (Exact uniform sampling with expected time). *For a satisfiable circuit, an exactly uniform satisfying assignment can be sampled with expected running time bounded by a polynomial times the exponential factor in (27), using independent fair random bits. Unsatisfiability is reported deterministically within the same counting bound.*

Proof. Compute Z . If $Z = 0$, report unsatisfiability. If $Z = 1$, unrank zero. For $Z > 1$, put $L = \lceil \log Z \rceil$. Draw L independent fair bits, interpreted as an integer $R \in \{0, \dots, 2^L - 1\}$. Reject and repeat when $R \geq Z$. Each trial accepts with probability $Z/2^L > 1/2$, so fewer than two trials are needed in expectation. Conditional on acceptance, every rank is equally likely. Unranking is a bijection, so every satisfying assignment has probability exactly $1/Z$. Since $L \leq u$, the expected random-bit and integer-processing cost is polynomial, and Corollary 8.3 bounds the deterministic remainder. $\square$

The sampler returns a solution with probability one and in finite expected time; a measure-zero random stream can reject forever. The expected-time qualifier is essential for this fair-bit implementation. With a fixed worst-case bound of L coin flips, every output probability is an integer multiple of 2^{-L} , even if some branches stop early. It cannot be exactly $1/3$ on each of three solutions. Allowing rejection, or allowing a declared failure outcome, resolves this elementary obstruction; silently assuming a unit-cost arbitrary biased coin does not.

Finally, a uniform satisfying *full assignment* need not give a uniform positive *projected input*. In the running example, projecting the uniform distribution on 010, 101, 110 onto x gives probabilities $1/3$ and $2/3$. Uniformity over witnesses weights each ordinary input by its number of witnesses. The algorithm here counts and samples full satisfying assignments; counting or sampling distinct projected inputs is a different task.

9 What the structural bounds do and do not force

9.1 What critical amplification would force

The coefficient $1/5$ comes from balancing three upper bounds. Near equality therefore forces several structural conditions simultaneously. The following statements concern a normalized circuit with $q \geq 1$ and a witness-dependent gate as output, using the notation of Section 7.

Lemma 9.1 (Finite stability of the three-way balance). *Fix $\delta > 0$, put $\alpha = 1/3 + \delta$, and choose $A_\delta \geq 1$ large enough for*

$$C(g) \leq p + A_\delta(q + 1)2^{\min\{h, b, \alpha\kappa\}}.$$

If $t \geq 0$ and $C(g) \geq p + A_\delta(q + 1)2^t$, then

$$\Delta = q + 1 - (2 + 1/\alpha)t \geq 0,$$

and there are $e_h, e_b, e_\kappa \geq 0$ satisfying

$$h = t + e_h, \quad b = t + e_b, \quad \kappa = t/\alpha + e_\kappa, \quad e_h + e_b + e_\kappa \leq \Delta.$$

Proof. Comparing the two size inequalities forces $h, b, \alpha\kappa \geq t$. Subtract these lower bounds from $h + b + \kappa \leq q + 1$. $\square$

This elementary lemma is useful because it preserves the fixed- δ constant before any limit is taken. Its asymptotic consequence also constrains the residual graph, beyond the original cycle count.

Theorem 9.2 (Necessary structure at the critical gate budget). *Consider a family with $m \rightarrow \infty$ declared witness bits, normalized size $s = p + q$, and*

$$s + 1 = (5 + o(1))m, \quad \log C(g) \geq m - o(m).$$

Let a be the number of used ordinary inputs. Then

$$\begin{aligned} p &= o(m), & q &= (5 + o(1))m, \\ a, h, b &= (1 + o(1))m, & \kappa &= (3 + o(1))m. \end{aligned}$$

For every choice of legal equality splitting and reductions in Lemma 7.2, the resulting kernel K is eventually nonempty and satisfies

$$\begin{aligned} |V(K)| &= (6 + o(1))m, & \kappa - \kappa(K) &= o(m), \\ \text{cw}(K) &= (1 + o(1))m, & \text{pw}(K) &= (1 + o(1))m. \end{aligned}$$

Here cutwidth and pathwidth mean their minimum over all orders and decompositions. No hard family satisfying the hypotheses is asserted.

Proof. Since $p = O(m)$ whereas $C(g) \geq 2^{m-o(m)}$, $\log(C(g) - p) \geq m - o(m)$. For each fixed $\delta > 0$, Theorem 7.6 therefore gives

$$h \geq m - o(m), \quad b \geq m - o(m), \quad \kappa \geq m/\alpha - o(m).$$

The prefactor contributes only $O_\delta(\log m)$ to the logarithm. Using $p + h + b + \kappa \leq s + 1$, we obtain

$$\limsup \frac{p}{m} \leq 3 - 1/\alpha, \quad \limsup \frac{h}{m}, \limsup \frac{b}{m} \leq 4 - 1/\alpha.$$

These inequalities hold for every fixed δ . Letting δ tend to zero after taking the limits proves $p = o(m)$ and $h/m, b/m \rightarrow 1$. The budget and the fixed- δ lower bounds then give $\kappa/m \rightarrow 3$, and $q/m \rightarrow 5$ follows from $s = p + q$.

To relate a to h , note first that $h \leq a + p$. In the undirected graph on the a ordinary inputs and p independent gates, every connected component contains an interface signal, because every vertex is in the output cone. If there are c components, then $c \leq h$, and counting edges gives $a + p - c \leq 2p$. Thus $a \leq p + h$, so $|a - h| \leq p$ and $a/m \rightarrow 1$.

Fix any sequence of legal reductions along the family. An empty kernel would give $C(g) = O(s)$, so eventually it is nonempty. Write $N = |V(K)|$. The finite compiler bound, applied to an order of minimum cutwidth, gives

$$C(g) \leq p + 400(q + 1)2^{\text{cw}(K)}, \quad \text{cw}(K) \geq m - o(m).$$

The cutwidth lower bound and $\text{cw}(K) \leq 3N/2$ imply $N \rightarrow \infty$. On the other hand $N \leq 2(\kappa - 1) \leq 6m + o(m)$. For every fixed $\eta > 0$, the cubic pathwidth theorem and Lemma 7.3 give, for sufficiently large N ,

$$\text{cw}(K) \leq \text{pw}(K) + 2 \leq (1/6 + \eta)N + 2.$$

Hence $\liminf N/m \geq 1/(1/6 + \eta)$ for every $\eta > 0$, proving $N/m \rightarrow 6$. The cubic identity $\kappa(K) = N/2 + 1$ gives the cycle loss. The same inequalities sandwich each width between $m - o(m)$ and $m + o(m)$. All constants depend only on the fixed slack, so this argument applies to any selected legal reduction sequence. $\square$

There is little unused fanin capacity in such a family. If r counts independent-signal input pins of the dependent gates, with multiplicity, and $U = 2q - (\ell + r)$, then

$$q + 1 - h - b - \kappa = U + (r - h) = o(m).$$

Both terms are nonnegative. Thus only $o(m)$ input slots are missing from binary gates, and ordinary signals have only $o(m)$ repeated uses at the interface. This concerns distinct signals, not semantic independence of their values.

Combining the structural and witness conditions. The hypotheses hold whenever $s + 1 = (5 + o(1))m$ and $C(g) \geq T = 2^m(s + 1)/L$ with $\log L = o(m)$. Corollary 10.7 then also forces $\Omega((T/n) \log(2 + T/n))$ positive ordinary inputs with $O(nL)$ declared witnesses. Removing the $m - b$ unused witness bits divides each positive fiber and this threshold by exactly 2^{m-b} ; the two witness conventions must be kept consistent. Enumeration already gives $m - b \leq \log L$ under this amplification hypothesis. These are simultaneous necessary conditions. Large width, balanced budgets, and sparse fibers alone do not establish a lower bound on $C(g)$.

9.2 Critical graph budgets can coexist with an easy projection

The critical gate and cycle budgets alone do not force a hard projection. A large class of cubic graphs can occur as exact residual kernels with nearly critical budgets and a simple projection. The distinction between the supplied circuit and a minimum circuit is essential here.

Theorem 9.3 (Cubic realization with controlled budgets). *Let K be a finite simple 2-vertex-connected cubic graph and put $k = |V(K)|/2 + 1$. Set $t = \lfloor k/3 \rfloor + 1$; when K is triangle-free, one may instead take $t = \lceil k/3 \rceil$. There is an acyclic AND-only verifier on t ordinary and t witness inputs with*

$$(p, q, h, b, \ell, \kappa) = (0, k + 2t - 1, t, t, 2k + 3t - 2, k).$$

It has no constants, unary gates, repeated input wires at a gate, unused inputs, or gates outside its output cone. A specified legal equality splitting and reduction sequence produces exactly K , with zero cycle-rank loss. Every input is essential, but

$$f(x, y) = \bigwedge_{i=1}^t x_i \wedge \bigwedge_{j=1}^t y_j, \quad g(x) = \bigwedge_{i=1}^t x_i, \quad C(f) = 2t - 1, \quad C(g) = t - 1.$$

Thus $h + b + \kappa = q + 1$ and $t = k/3 + O(1)$, while the supplied circuit exceeds its minimum size by exactly k gates.

The syntactic conditions in the statement are those used by the compiler's normalization. They do not prohibit identities involving several gates, such as associativity and idempotence of AND. We give the construction and its accounting in full.

Proof. First give K an acyclic orientation with one source and one sink. This is the classical st -numbering property of 2-connected graphs [13]. An existence proof suffices here. Start with a cycle, choose adjacent vertices s, z , and order the vertices along the s -to- z path obtained by deleting the edge sz . If some vertices remain outside the current set, one of their components has two distinct attachments to the current set; otherwise its unique attachment would be a cut vertex. Choose a path through that component between two such attachments u, v , with internal vertices outside. Orient the path from its earlier to its later endpoint and insert its internal vertices, in order, immediately after the earlier endpoint. Every old internal vertex retains neighbors on both sides, and each new vertex has its two path neighbors on opposite sides. Repeat until all vertices are ordered and orient every edge forward. Only s is a source and only z is a sink.

The circuit skeleton. Every other vertex is a fork with indegree one and outdegree two, or a join with indegree two and outdegree one. If their numbers are F, J , the degree sum gives

$$3 + F = J + 3, \quad F + J + 2 = 2(k - 1), \quad F = J = k - 2.$$

Put witness y_1 at s . A fork copies its incoming wire at no gate cost. A join ANDs its incoming wires. At z , two AND gates combine the three incoming wires. This skeleton has k gates. Temporarily assign each join output its own formal wire name, even if that join receives the same name twice. Call such a join a conflict.

The sink receives at least two different names. Indeed a topologically last join exists, and any path from it to the sink contains only forks. If its output passes through a nonempty fork tree, a last fork would have both outgoing edges to the sink, contradicting simplicity. Its output therefore reaches the sink directly and on just one edge. It cannot name all three sink inputs. Choose two distinct names for the first sink gate; its new output and the third original input are distinct at the second gate.

The conflict budget. The subgraph on the source and forks is a forest. A cycle with an acyclic orientation would have a vertex with two incoming edges, which these vertices cannot have. Its components are exactly the trees carrying one formal wire name. A component not containing s with u forks has one incoming edge and $u + 1$ outgoing edges. Each conflict it supplies uses two outgoing edges. For $u = 1$ there is no conflict, by simplicity; for $u \geq 2$ their number is at most

$$\left\lfloor \frac{u + 1}{2} \right\rfloor \leq \frac{2u}{3}.$$

The inequality is checked directly at $u = 2$ and follows from $(u + 1)/2 \leq 2u/3$ for $u \geq 3$.

The source component with u additional forks has $u + 3$ outgoing edges. At $u = 0$ it has no conflict; at $u = 1, 2$ it has at most two. For $u \geq 3$ its number of conflicts is at most $\lfloor (u + 3)/2 \rfloor \leq 2u/3 + 4/3$. The latter bound covers the small cases as well. Summing over all $F = k - 2$ forks bounds the total number R of conflicts by

$$R \leq \lfloor 2k/3 \rfloor.$$

If K is triangle-free, the $u = 1$ source component has no conflict, since one would make a triangle through the source, fork, and target join. The source bound improves to $2u/3 + 1$, giving

$$R \leq \lfloor (2k - 1)/3 \rfloor \quad \text{in the triangle-free case.}$$

There are $2t - 1$ unused inputs available: the t ordinary inputs and $t - 1$ additional witnesses. The chosen t ensures $R \leq 2t - 1$ in each case. On one incoming edge of each conflicting join insert

$$\text{new wire} = \text{old wire} \wedge \text{fresh input},$$

using a different input each time. Insert any remaining inputs by such gates in a chain on one original edge. The fresh output names split copied wires; they never identify distinct old names. Hence all conflicts are repaired without creating new ones, including at the two sink gates. The orientation gives a topological gate order.

The verifier budget and its function. Each attachment has a witness-dependent old input, so all gates are witness-dependent. There are $k + 2t - 1$ gates, every ordinary input appears at one gate pin, and all t witnesses are used. Therefore $p = 0$, $h = b = t$, and

$$\ell = 2q - t = 2k + 3t - 2, \quad \kappa = \ell - q - b + 1 = k.$$

Every vertex reaches the unique sink. ANDing along the circuit thus gives the conjunction of all inputs, proving the displayed f, g . Each input is essential by setting all other inputs to one. The effective-input lower bound in Lemma 6.1, met by an AND chain, gives $C(f) = 2t - 1$ and $C(g) = t - 1$.

The exact residual kernel. Form the actual consistency incidence graph. A wire with u forks after its defining join has $u + 1$ uses and one defining incidence. Choose its equality splitting to be precisely that fork tree: an equality of degree $u + 2$ is replaced by u ternary equalities. For $u = 0$, keep its degree-two equality. The source wire with u forks has $u + 3$ uses; choose its splitting into the source and those u forks. Attachments divide these trees into smaller pieces, to which the same construction applies. Reversing these splittings contracts each equality tree to the actual incidences of one circuit wire, verifying that they are permitted choices in Lemma 7.2.

Contract the output-one assertion through the output equality into the second sink gate. That constraint now has two ports. Contract the degree-two equality between the sink gates and then the remaining two-port constraint into the first sink gate. Its surviving table is the conjunction of the three original incoming bits, at vertex z . For each additional witness, contract its degree-one equality into its attachment constraint, leaving two ports. For example, $\bigvee_y [v = u \wedge y] = [v \leq u]$: the resulting table is retained, not replaced by an equality. Ordinary attachments already have two ports, since their ordinary inputs are coefficients. Contract these two-port constraints and all degree-two wire equalities along their original edges. Their updated neighboring tables retain the semantic effects of the attachments.

Topologically these operations undo edge subdivisions, the sink tree, and attached leaf trees. They leave exactly the marked vertices and edges of K . No loop or parallel-edge reduction is needed: every step is a permitted single-edge contraction and preserves cycle rank. Since K is simple and cubic, the reduction terminates there, with the stated zero loss and the linear operation cost of the compiler. $\square$

For triangle-free K with $k = 3t$, the parameters are exactly

$$(p, q, h, b, \ell, \kappa) = (0, 5t - 1, t, t, 9t - 2, 3t).$$

Thus acyclicity, essential inputs, the critical budget, and zero cycle loss do not prevent these kernel topologies. The theorem asserts existence of the specified reduction, not that every reduction must return K . It does not realize the hardness hypothesis of Theorem 9.2: its supplied circuit is nonminimal and its projection is easy. Stronger semantic reductions or choosing a smaller equivalent verifier can remove the apparent structural difficulty.

10 Many witnesses give small nonuniform circuits

For x with $g(x) = 1$, define its witness set $W_x = \{y : f(x, y) = 1\}$. The minimum size of a set $H \subseteq \{0, 1\}^m$ intersecting every nonempty W_x is a natural coverage parameter, say $\tau(f)$. For $g \equiv 0$,

take $\tau(f) = 0$. By evaluating only witnesses in H ,

$$C(g) \leq \tau(f)s + \max\{\tau(f) - 1, 0\}. \quad (28)$$

Proposition 10.1 (Witness-density bound). *Suppose $n \geq 1$ and every nonempty W_x has size at least K , and set $\rho = K/2^m > 0$. There is a fixed set of at most*

$$T = \min \left\{ 2^m, \left\lceil \frac{n \ln 2 + 1}{\rho} \right\rceil \right\} \quad (29)$$

witnesses meeting all nonempty W_x . Hence

$$C(g) \leq T(s + 1) - 1 = O((s + 1) \min\{2^m, n2^m/K\}). \quad (30)$$

For the identically zero function, the conclusion holds trivially.

Proof. Take $t = \lceil (n \ln 2 + 1)/\rho \rceil$ independent uniform witnesses. A fixed nonempty witness set is missed with probability at most $(1 - \rho)^t \leq e^{-\rho t}$. A union bound over at most 2^n ordinary inputs bounds the chance of any miss by $e^{-1} < 1$. Thus a successful sample exists. Remove duplicates, or use all 2^m witnesses if that is smaller. Hardwire the resulting set. $\square$

The same witnesses work for *all* inputs; this is why the result gives one deterministic circuit. Finding the set efficiently is not part of the claim. If every positive input has an inverse-polynomial witness density and s is polynomial in n , then g has polynomial-size circuits.

For a blowup within a factor L of $2^m(s + 1)$, this construction requires $K = O(nL)$ as a necessary condition. This is a condition on the *minimum* positive fiber size. It does not say that every positive input must have few witnesses; a small exceptional collection of inputs can determine exact circuit complexity. Conversely, rare witnesses do not prove hardness: $f(x, y) = [y = 0^m]$ has one witness for every x , yet its projection is constant.

10.1 Cover only what is expensive to correct

Throughout the coverage refinements below, assume $n \geq 2$.

The coverage bound charges s gates for every witness, while the support bound compiles every possible restriction at every gate. Both charges can be avoided simultaneously. A selected witness family may induce very few different restrictions at expensive gates. Moreover, the family need not cover every positive input if the remaining exceptions have a cheap circuit.

We first record a convenient elementary synthesis bound. Its role here is to price the correction, not to claim an optimal theorem about sparse Boolean functions; sharper results in some ranges are discussed in Section 15.

Lemma 10.2 (A circuit for a finite exceptional set). *For $n \geq 2$, the characteristic function of any R points of $\{0, 1\}^n$ has complexity*

$$O\left(\frac{nR}{\log(R + 2)}\right).$$

The expression is zero for $R = 0$. A simpler bound is nR .

Proof. A single point indicator takes $n - 1$ gates over B_2 : test the first two prescribed bits with one binary gate, then AND each subsequent prescribed literal into the result. ORing R indicators proves the simpler bound and handles $R \leq 1$.

For $R \geq 2$, put $k = \min\{n, \lfloor \log R \rfloor\}$ and partition the input coordinates into $\lceil n/k \rceil$ blocks of at most k bits. Within each block compute every assignment indicator. Sharing prefix indicators makes this library cost $O(2^k)$ gates, rather than $O(k2^k)$. Each of the R selected points is an AND of one indicator from each block. OR these point indicators. The total cost is

$$O\left(\left\lceil \frac{n}{k} \right\rceil (2^k + R)\right) = O\left(\frac{nR}{\log(R+2)}\right).$$

□

Fix a particular verifier circuit. For a witness set H , let

$$N_v(H) = |\{y|_{Y_v} : y \in H\}|, \quad R(H) = |\{x : g(x) = 1, H \cap W_x = \emptyset\}|.$$

The output has its own support Y_{out} ; if the output is a gate, its occurrence in an output term below is in addition to its occurrence in the sum over gates.

Lemma 10.3 (Selected restrictions and exact correction). *For every $H \subseteq \{0, 1\}^m$, including $H = \emptyset$,*

$$C(g) \leq \sum_v N_v(H) + N_{\text{out}}(H) + nR(H), \quad (31)$$

$$C(g) \leq \sum_v N_v(H) + N_{\text{out}}(H) + O\left(\frac{nR(H)}{\log(R(H)+2)}\right). \quad (32)$$

The sums run over the gates of the supplied verifier.

Proof. Copy gate v once for each restriction to Y_v that occurs in H . An ordinary predecessor is the same input in every copy. A witness predecessor becomes its fixed bit. A gate predecessor uses its copy indexed by the restriction to its own, smaller support. This computes every selected $f(x, y)$ with the indicated sharing.

Deduplicate the output-support restrictions before taking their OR; this costs at most $\max\{N_{\text{out}}(H) - 1, 0\}$ gates. Its output g_H never falsely accepts an input. Let e_H be the characteristic function of the $R(H)$ positive inputs missed by H . Then $g = g_H \vee e_H$. Point indicators with their final ORs cost at most $nR(H)$, proving (31). Use Lemma 10.2 for (32); the final OR is absorbed when $R(H) > 0$ and is unnecessary otherwise. For empty H only the correction is needed. □

10.2 An occupancy–alteration theorem

Let μ be any probability distribution on witness strings. It is chosen separately for each verifier and need not be efficiently samplable. Write

$$p_{v,\alpha} = \mathbb{P}_{y \sim \mu}[y|_{Y_v} = \alpha], \quad p_x = \mu(W_x).$$

For an integer $t \geq 0$, define

$$B_\mu(t) = \sum_v \sum_{\alpha \in \{0,1\}^{Y_v}} (1 - (1 - p_{v,\alpha})^t) + \sum_{\alpha \in \{0,1\}^{Y_{\text{out}}}} (1 - (1 - p_{\text{out},\alpha})^t), \quad (33)$$

$$E_\mu(t) = \sum_{x: g(x)=1} (1 - p_x)^t. \quad (34)$$

Here $\{0, 1\}^Y$ means assignments to the coordinate set Y . At $t = 0$ every power is one, including 0^0 in these sampling expressions. Thus $B_\mu(0) = 0$ and $E_\mu(0) = |g^{-1}(1)|$.

Theorem 10.4 (Joint sharing, sampling, and alteration). *For $n \geq 2$, every μ and every integer $t \geq 0$ give*

$$C(g) \leq B_\mu(t) + nE_\mu(t), \quad (35)$$

$$C(g) \leq B_\mu(t) + O\left(\frac{nE_\mu(t)}{\log(E_\mu(t) + 2)}\right). \quad (36)$$

The constant in the second bound is absolute. Both conclusions concern the existence of one exact circuit, with no error on any input.

Proof. Draw t independent witnesses from μ and let H contain the distinct sampled strings. A support assignment α appears at gate v with probability $1 - (1 - p_{v,\alpha})^t$. Consequently

$$\mathbb{E}\left[\sum_v N_v(H) + N_{\text{out}}(H)\right] = B_\mu(t).$$

A positive input x is missed with probability $(1 - p_x)^t$, so $\mathbb{E}R(H) = E_\mu(t)$. Take expectations in (31) and choose a sample whose construction cost is no greater than its mean. This proves (35).

For the sparse bound, the function $\phi(z) = z/\log(z+2)$ is concave on $[0, \infty)$. In natural logarithms its second derivative has the sign of $2z - (z+4)\ln(z+2)$, which is negative: the opposite quantity is positive at zero and has positive derivative $\ln(z+2) - 1 + 2/(z+2)$. Jensen's inequality gives $\mathbb{E}\phi(R(H)) \leq \phi(\mathbb{E}R(H))$. Apply this inequality to (32) and again select a sample of cost at most the expectation. If $E_\mu(t) = 0$, there are no exceptions with positive probability and no correction cost. No description of μ is placed in the resulting circuit; only the selected witnesses and the correction are hardwired. $\square$

The two terms describe different resources. $B_\mu(t)$ counts distinct local computations, while $E_\mu(t)$ counts positive inputs still unexplained by the selected witnesses. In particular, a large t need not imply a large circuit: repeatedly sampling restrictions that have already occurred adds no new local computation.

Corollary 10.5 (Density and sharing, with a minimum at each gate). *Suppose $p_x \geq \rho > 0$ for every positive input. Set $t = \lceil (n \ln 2 + \ln 4)/\rho \rceil$. There is a witness cover H whose shared-restriction circuit has size at most $2B_\mu(t)$. For uniform witnesses this gives*

$$C(g) = O\left(\sum_v \min\{2^{|Y_v|}, n/\rho\} + \min\{2^{|Y_{\text{out}}|}, n/\rho\}\right).$$

Proof. A union bound gives $\mathbb{P}[R(H) > 0] \leq 2^n e^{-\rho t} \leq 1/4$. If $B_\mu(t) > 0$, Markov's inequality bounds the probability that the occupancy cost exceeds $2B_\mu(t)$ by $1/2$. Some sample therefore covers every positive input and has the claimed cost. The case of no positive inputs is immediate. For a nonempty projection and $t \geq 1$, the output occupancy has expectation at least one, so the remaining case $B_\mu(t) = 0$ cannot arise. Each occupancy is at most both its number of possible support assignments and the sample count t . $\square$

Unlike taking the minimum of two total-cost estimates, this bound takes a separate minimum at each gate. Section 11 gives a family where this distinction makes an exponential difference.

10.3 Hardness requires a population of sparse fibers

Corollary 10.6 (The entire lower tail of the witness count matters). *For $1 \leq K \leq 2^m$, put $R_K = |\{x : 0 < |W_x| < K\}|$. Then*

$$C(g) = O\left(\frac{n(s+1)2^m}{K} + \frac{nR_K}{\log(R_K + 2)}\right).$$

Proof. Use the proof of Proposition 10.1 only on inputs with at least K witnesses. A set of $O(n2^m/K)$ witnesses covers all such inputs. Every missed positive input belongs to the R_K -point exceptional set. Evaluate the selected witnesses and use Lemma 10.2 to correct precisely those that were missed. The function $z/\log(z+2)$ is increasing, so the correction cost is bounded by the displayed expression. $\square$

Corollary 10.7 (A quantitative sparse-fiber population). *Let $T = 2^m(s+1)/L$, where $L \geq 1$, and assume $C(g) \geq T$. There are absolute constants $c, c' > 0$ such that*

$$|\{x : 0 < |W_x| \leq cnL\}| \geq c' \frac{T}{n} \log\left(2 + \frac{T}{n}\right).$$

Proof. Choose a sufficiently large absolute c . If $K = \lceil cnL \rceil \leq 2^m$, the first term of the profile bound is at most $T/2$ after increasing c to account for its absolute constant. It follows that $R_K/\log(R_K + 2) \geq c_0 T/n$ for an absolute $c_0 > 0$. This implies $R_K \geq c_1 T/n$, and substitution back into the logarithm gives $R_K \geq c_2 (T/n) \log(2 + T/n)$. Adjusting c includes the ceiling in the asserted threshold. If the chosen threshold exceeds 2^m , all positive inputs qualify. Apply Lemma 10.2 directly to the support of g and use the same inversion. $\square$

The original minimum-density condition asserts that at least one positive input has few witnesses. This corollary forces a number of such inputs controlled by the claimed hardness. It remains a necessary condition: even a large set of unique-witness inputs can have an easy projection. The stronger theorem 10.4 also detects cover geometry that a list of fiber cardinalities cannot describe.

Consistency obstructs naive aggregation. The shared-witness counterexample in Section 1 shows why existence cannot be pushed independently through an AND gate. Lemma 6.3 works because it proves that the witness sets being combined are disjoint. The cut construction restores that condition while retaining the defining equations. This is also why identities that compress parity aggregates do not automatically compress existential OR.

11 Worked families: what the parameters reveal

The following examples distinguish the guarantees of compilation methods. Their projected functions are explicitly easy. We calculate their actual complexities where possible, so that a large upper-bound expression is never mistaken for a lower bound on the function.

11.1 An exponential improvement from selecting and sharing together

Let $N \geq 2$, $k \geq 3$, and $\ell \geq 2$. The ordinary inputs are $z \in \{0, 1\}^N$, $u \in \{0, 1\}^k$; the witnesses are $a \in \{0, 1\}^k$, $b \in \{0, 1\}^\ell$. Indices on a are cyclic. Define

$$T(u, a) = \bigwedge_{i=1}^k [u_i = a_i \oplus a_{i+1}], \quad (37)$$

$$f(z, u; a, b) = \text{PAR}_N(z) \wedge T(u, a) \wedge \text{PAR}_\ell(b). \quad (38)$$

Use XOR chains for the two parities, one XOR and one XNOR for each equation, a prefix AND chain for the equations, then AND the z parity with T and finally with the b parity. This supplied circuit has

$$n = N + k, \quad m = k + \ell, \quad s = N + 3k + \ell - 1.$$

Every gate is in the output cone. Every input is essential: starting from an accepting assignment, changing a z or b bit flips a required parity; changing a u bit violates an equation; changing an a bit violates two equations. Therefore

$$N + 2k + \ell - 1 \leq C(f) \leq N + 3k + \ell - 1.$$

The supplied size is within k gates of the elementary lower bound; there is no padding or dead subcircuit.

XORing the equations shows that T is satisfiable only if $\text{PAR}_k(u) = 0$. Conversely, a choice of a_1 determines all other a bits along the first $k - 1$ equations, and the last holds precisely when that parity is zero. Hence

$$g(z, u) = \text{PAR}_N(z) \wedge \neg \text{PAR}_k(u), \quad C(g) = N + k - 1. \quad (39)$$

The upper bound uses two parity chains and one binary gate; the lower bound follows because all $N + k$ ordinary inputs are essential.

Each positive input has two complementary choices of a and $2^{\ell-1}$ odd-parity choices of b , giving exactly 2^ℓ witnesses. The fibers for distinct even-parity u are disjoint. Fix one odd-parity b_0 and select

$$H = \{(a, b_0) : a_1 = 0\}, \quad |H| = 2^{k-1} =: h.$$

Exactly one member of H works for each even-parity u , so $\tau(f) = h$. With $A(H) = \sum_v N_v(H) + N_{\text{out}}(H) - 1$, the precise support counts for the supplied circuit are as follows.

Circuit part	Selected H copies	All support assignments
z parity	$N - 1$	$N - 1$
k XORs and k XNORs	$8k - 8$	$8k$
Prefix conjunction	$3 \cdot 2^{k-1} - 4$	$3 \cdot 2^k - 8$
b parity	$\ell - 1$	$2^{\ell+1} - 4$
AND of z parity and T	2^{k-1}	2^k
Final AND	2^{k-1}	$2^{k+\ell}$
Output OR	$2^{k-1} - 1$	$2^{k+\ell} - 1$

For the second row, two cycle edges touch a_1 . Their gates each have two selected assignments, while the remaining edges each have four. For the third row, the j th prefix, $2 \leq j < k$, uses

$a_1, \dots, a_{j+1}$; the last uses all a bits. Thus its selected sum is $\sum_{j=2}^{k-1} 2^j + 2^{k-1}$, and the full sum doubles it. The parity-chain counts sum $2^2, \dots, 2^\ell$. Adding the rows gives

$$\begin{aligned} A(H) &= N + \ell + 8k + 3 \cdot 2^k - 15, \\ A(\text{full}) &= N + 8k + 4 \cdot 2^k + 2^{\ell+1} + 2^{k+\ell+1} - 14. \end{aligned}$$

Exactly the k witness vertices a_i have fan-out two; all gate outputs have fan-out one. Hence the original shared-vertex count is $r = k$, and $s + 1 - n - m = k$.

Now set $N = 2^k$ and $\ell = k$. The comparison becomes

Quantity	Order as $k \rightarrow \infty$
Actual $C(g)$ and verifier size	$\Theta(2^k)$
Selected and shared cover $A(H)$	$\Theta(2^k)$
Independent optimal-cover estimate $h(s + 1) - 1$	$\Theta(4^k)$
Full-support estimate $A(\text{full})$	$\Theta(4^k)$
Shared-vertex estimate $(s + 1)2^r$	$\Theta(4^k)$
Uniform-density and exhaustive estimates	$\Theta(8^k)$

The universal synthesis ceiling is still larger, and the original three-way exponent has $\min\{n, m, s + 1 - n - m\} = k$. Thus simultaneous selection and sharing improve the minimum of all those original guarantee expressions by a factor $\Theta(2^k)$. This does not say that running an old compiler and then semantically simplifying it requires $\Theta(4^k)$ gates. The new consistency-graph theorem can also recognize this family's easy projection.

11.2 Even every row mass leaves an exponential ambiguity

For the same diagonal family, let μ be uniform on H . Every positive row has mass $1/h$, and

$$E_\mu(t) = 2^{n-2}(1 - 1/h)^t, \quad B_\mu(t) \leq A(H) + 1.$$

Taking $t = \lceil h((n - 2) \ln 2 + \ln n + 1) \rceil$ makes $nE_\mu(t) \leq e^{-1}$. Although $t = \Theta(4^k)$, the bound in (35) is $O(2^k)$: repeated local restrictions do not add gates.

Instead let ν be uniform on all witnesses whose b parity is odd. *Every positive row still has exactly the same mass $1/h$.* Both distributions maximize the minimum row mass, since the h distinct positive fibers are disjoint. Nevertheless

$$\inf_{t \geq 0} \{B_\mu(t) + nE_\mu(t)\} = \Theta(2^k), \quad \inf_{t \geq 0} \{B_\nu(t) + nE_\nu(t)\} = \Theta(4^k), \quad (40)$$

where the infima are over integer sample counts.

Here is a direct proof of the second assertion. There are $M = 2^{2k-1}$ equally likely witnesses under ν , all distinguished by the output support. Thus $B_\nu(t) \geq M(1 - (1 - 1/M)^t)$. If $t \geq M/8$, this is at least $M(1 - e^{-1/8})$. If $t < M/8$, use $\ln(1 - p) \geq -p/(1 - p)$ with $p = 1/h \leq 1/4$ to obtain

$$E_\nu(t) \geq 2^{n-2}e^{-N/6}.$$

This eventually exceeds every constant multiple of 4^k , since $n = N + k$ and $N = 2^k$. These cases prove the lower bound on the *guarantee expression*. Its upper bound follows from the full support count and $E_\nu(t) \rightarrow 0$. The first assertion follows from the preceding choice of t and the actual lower bound on $C(g)$. The same exponential distinction survives the sparse correction term, because $E/\log(E + 2)$ in the second case is still exponentially large in N .

Thus even the whole vector $(\mu(W_x))_{x:g(x)=1}$ does not determine the cost of this compiler. How the same witnesses collide on internal supports is additional information.

11.3 Better coverage can be a worse circuit choice

For $L \geq 8$, put $n = 2L$, $m = 2$, and build

$$f_L(x; y) = (y_1 \oplus x_1 \oplus \cdots \oplus x_L) \vee (y_2 \oplus x_{L+1} \oplus \cdots \oplus x_{2L}),$$

with each XOR chain starting at its witness bit. There are L gates with each singleton witness support and one with both witnesses. All inputs are essential, proving $C(f_L) = 2L + 1$ by the input bound. Its projection is nonetheless $g = 1$.

Each row excludes just one of the four witnesses, and each excluded label occurs equally often. Therefore $\min_x \lambda(W_x) = 1 - \max_y \lambda(y)$, whose unique maximizing distribution is the uniform distribution u , with value $3/4$. For $F_\lambda(t) = B_\lambda(t) + nE_\lambda(t)$, direct occupancy gives

$$B_u(t) = 4L(1 - 2^{-t}) + 8(1 - (3/4)^t), \quad E_u(t) = 2^{2L-2t}.$$

For $t < L$ the correction is at least $8L$; for $t \geq L$ the occupancy is at least $4L(1 - 2^{-L})$. Hence $F_u(t) \geq 4L(1 - 2^{-L})$ for every integer $t \geq 0$.

Let v instead give equal mass to 00 and 01. Its minimum row mass is only $1/2$. For $t \geq 1$,

$$B_v(t) = L + (2L + 4)(1 - 2^{-t}), \quad E_v(t) = 2^{2L-1-t}.$$

At $t = 2L + \lceil \log(2L) \rceil$ this gives

$$F_v(t) \leq 3L + 4.5 < 4L(1 - 2^{-L}) \leq \inf_j F_u(j).$$

The unique coverage optimizer is therefore worse for the circuit-cost objective. The asymptotic comparison is $3L + O(1)$ versus $4L + O(1)$. It also survives any fixed positive constant multiplying the sparse correction: for $t < L/2$ that correction grows faster than L , while for $t \geq L/2$ the uniform occupancy is already $4L - o(1)$.

This objective belongs to the supplied circuit representation. Moving the witnesses to the ends of the XOR chains changes its supports without changing f_L or the optimal verifier size. Its constant projection also emphasizes that an obstruction to a particular compilation strategy is not a circuit lower bound.

11.4 Two additional cautions

Fiber sizes do not determine covers. For ordinary $u \in \{0, 1\}^k$ and witnesses $(c, v) \in \{0, 1\} \times \{0, 1\}^k$, consider rows

$$W_u^A = \{(0, 0^k), (1, u)\}, \quad W_u^Q = \{(0, u), (1, \neg u)\}.$$

Both profiles consist entirely of size-two fibers; both have linear-size verifiers with all inputs essential. The first has a common witness and cover number one; the second partitions the witness universe and has cover number 2^k .

Nor may each gate choose its own best cover. In the relation $f(x_1, x_2; y_1, y_2) = [x \neq y]$, any two distinct witnesses cover every input. The set $\{00, 01\}$ uses only one y_1 value, while $\{00, 10\}$ uses only one y_2 value. No cover achieves both simultaneously, because it would fix both bits. The minimum of the sum of these two occupancies is three, whereas the sum of their separate minima is two. The occupancy theorem uses one common distribution and one common sample throughout the circuit.

12 What random verifiers do, and do not, demonstrate

12.1 Uniform truth tables usually project to a dense function

Choose every value of f independently and uniformly from $\{0, 1\}$. For each x ,

$$\mathbb{P}[g(x) = 0] = 2^{-2^m}. \quad (41)$$

Different rows are independent. Thus the number Z of zeros of g has exactly the distribution

$$Z \sim \text{Bin}(2^n, 2^{-2^m}), \quad \mathbb{E}Z = 2^{n-2^m}, \quad \mathbb{P}[g \equiv 1] = (1 - 2^{-2^m})^{2^n}. \quad (42)$$

In particular,

$$\mathbb{P}[g \neq 1] \leq 2^{n-2^m}.$$

If $2^m - n \rightarrow +\infty$, then $g \equiv 1$ with probability tending to one. If $n - 2^m \rightarrow +\infty$, then $\mathbb{P}[g \equiv 1] \rightarrow 0$. The constant-function transition is therefore at $2^m \approx n$, or $m \approx \log n$, with the exact probability in (42) accounting for integer effects.

Almost all such f have near-maximal complexity by the sharp counting theorem. Hence near-maximal verifiers with constant projection are abundant in this range. This is a statement about uniform random truth tables; it is not a distribution on small verifier circuits.

At the coarser scale of being within a constant factor of maximal complexity, even that scale alone does not fix $C(g)$. For $n \geq 3$ and $m \geq 1$, write $x = (a, u)$ and $y = (b, z)$. Choose functions $h(u, z)$ and $q(u)$ of complexities $\Theta(M_{n+m-2})$ and $\Theta(M_{n-1})$, respectively, and put

$$f((a, u), (b, z)) = (a \wedge (b \vee h(u, z))) \vee (\neg a \wedge q(u)).$$

Restriction to $a = 1, b = 0$ recovers h , while $g(a, u) = a \vee q(u)$ recovers q at $a = 0$. Upper bounds from the displayed circuits show $C(f) = \Theta(M_{n+m})$ and $C(g) = \Theta(M_n)$. In contrast, choose $h'(x, z)$ with complexity $\Theta(M_{n+m-1})$; then $b \vee h'(x, z)$ has verifier complexity $\Theta(M_{n+m})$ and constant projection. These examples concern constant-factor maximality, not a sharp $(1 - o(1))M_{n+m}$ requirement.

12.2 Balancing the projection does not make the verifier small

If instead each entry of f is independently one with probability

$$p_m = 1 - 2^{-1/2^m},$$

then $(1 - p_m)^{2^m} = 1/2$, so the projected function g is *exactly* a uniformly random Boolean function on n inputs. For large m , $p_m \sim (\ln 2)2^{-m}$. This ensemble corrects the density bias of uniform relations, but does not give a small verifier. The following counting fact makes the limitation precise.

Proposition 12.1 (Almost all functions need large nondeterministic circuits). *Even allowing an arbitrary number of witness bits, a uniformly random n -input function has minimum verifier complexity $\Omega(2^n/n)$ with probability $1 - o(1)$.*

Proof. By Lemma 6.1, an output cone of size at most s uses at most $s + 1$ witness bits, up to relabeling. Circuits with zero gates are also covered by this bound. Pad to s gates and name the witnesses from a fixed set of $s + 1$ names. The number of possible projected functions is at most

$$16^s (n + 2s + 3)^{2s+1} = 2^{O(s \log(n+s+2))}. \quad (43)$$

Each circuit determines only one projected function. For a sufficiently small constant $c > 0$, setting $s = c2^n/n$ makes the logarithm of (43) at most $(1 - \eta)2^n$ for some fixed $\eta > 0$ and all sufficiently large n , taking the integer part of $c2^n/n$ if needed. Divide by 2^{2^n} . $\square$

It follows that the balanced-projection ensemble has $C(f) = \Omega(2^n/n)$ with high probability. Randomizing a truth table can give a hard projection, but this method does not reach the small-verifier regime under discussion.

More generally, the ordinary counting comparison cannot by itself turn s gates into a $2^m s$ lower bound: the entire family of size- s verifiers has description entropy at most $O(s \log(n + s + 2))$, independently of the nominal witness count. Its projections might still contain very hard functions. Establishing that requires information about which functions the family contains, beyond this cardinality estimate.

13 Unconditional lower bounds and their limits

Proposition 13.1 (A baseline lower bound). *If $2 \leq s \leq M_n$, then*

$$D(n, m, s) > \frac{s-1}{2}.$$

If $s \geq M_n$, then $D(n, m, s) = M_n$. Thus, away from degenerate small parameters, $D(n, m, s) = \Omega(\min\{s, M_n\})$.

Proof. Take a circuit for a function of complexity M_n . Examine its gates in topological order and choose the first whose computed function has complexity greater than $(s-1)/2$. Such a gate exists, since the output does. Each of its predecessor functions has complexity at most $(s-1)/2$; inputs and constants have complexity zero. Computing these functions separately and applying the gate shows that the selected function has complexity at most s . Use that function as a verifier independent of y . For $s \geq M_n$, use a function attaining M_n . $\square$

This proves a lower bound without any nondeterministic amplification. The gap between it and the upper envelope can be exponential in m . It is not closed by Shannon counting.

Results in the literature also show that nondeterminism can sometimes save gates. Over $U_2 = B_2 \setminus \{\text{XOR}, \text{XNOR}\}$, Morizumi gives an n -input function with nondeterministic complexity at most $2n + o(n)$ and deterministic complexity $3n - o(n)$ [25, Theorem 3]. The function is an OR of parity functions on approximately $\sqrt{n}$ disjoint blocks. Its verifier guesses a block using about $\frac{1}{2} \log n$ bits, selects the block, and computes its parity. The result establishes a constant-factor advantage in that basis, far smaller than the full enumeration factor. The exact constants do not transfer to B_2 .

In the other direction, parity itself requires exactly $3(n-1)$ gates in both deterministic and nondeterministic U_2 circuits [24]. Neither result establishes an exponential deterministic/nondeterministic gap for unrestricted circuits.

13.1 The polynomial-verifier range contains the central open problem

Proposition 13.2 (The nonuniform complexity-class boundary). *The following statements are equivalent:*

- (i) $\text{NP} \subseteq \text{P/poly}$.
- (ii) *Every polynomial-size verifier family with polynomially many witness bits has a polynomial-size deterministic projection family, even if the verifiers themselves are nonuniform.*
- (iii) *For every polynomially bounded $m(n), s(n)$, the function $D(n, m(n), s(n))$ is polynomially bounded in n .*

Proof. If SAT has polynomial-size circuits, encode a verifier circuit together with its ordinary input as an instance of circuit satisfiability. The encoding length is polynomial in $n + m + s$. A polynomial-size SAT circuit, with the verifier description hardwired, computes the projection. This proves (i) implies (ii) and supplies a uniform polynomial size bound for each choice of polynomial parameter bounds, giving (iii).

Statement (iii) immediately implies (ii). Every language in NP has polynomial-size verifier circuits and polynomial witness length, so (ii) implies (i). $\square$

Consequently, a superpolynomial lower bound for the projection of a polynomial-size verifier family would prove $\text{NP} \not\subseteq \text{P/poly}$, even without a uniform construction of that family. This is stronger than $\text{P} \neq \text{NP}$ and remains open. The lower bound $C(g) \geq 2^{m-o(m)}C(f)$ asks for much more quantitative information.

If $s = \text{poly}(n)$ and $m = O(\log n)$, exhaustive expansion already gives polynomial size. A superpolynomial lower bound at every sufficiently large length therefore requires $m = \omega(\log n)$. For example, with $s = n^a$ and $m = \beta n$, $0 < \beta < 1$, the ceiling permits $2^{\beta n} n^a$ for large n . Proving a corresponding exponential lower bound for a verifier family is far beyond the unconditional baseline above.

14 A precise conditional version of exponential hardness

The Exponential Time Hypothesis (ETH) and Strong Exponential Time Hypothesis (SETH) are hypotheses about algorithms. A lower bound on the minimum size of a nonuniform circuit requires an appropriate circuit hypothesis, or an additional argument connecting the two. We explicitly use the nonuniform version of SETH, as formulated, for example, in Aggarwal et al. [1, Definition 2.9]:

For every $0 < \varepsilon < 1$, there is a fixed k such that no circuit family of size $O(2^{(1-\varepsilon)m})$ decides satisfiability of k -CNFs on m variables.

Here is an explicit verifier making the parameters transparent. List all k -clauses on distinct variables from $y_1, \dots, y_m$, including all sign choices. Their number is

$$n = 2^k \binom{m}{k} = \Theta_k(m^k).$$

Let x_c indicate whether clause c is present, and set

$$f(x, y) = \bigwedge_c (\neg x_c \vee c(y)). \quad (44)$$

Then $g(x)$ is satisfiability of the indicated formula. Computing all clauses, testing the indicators, and taking their conjunction uses $O_k(n + m)$ gates. Every indicator and every witness variable is essential when $m \geq k \geq 1$, so Lemma 6.1 also gives

$$s = C(f) = \Theta_k(n + m) = \Theta_k(m^k). \quad (45)$$

This mask encoding is polynomially interchangeable, for fixed k , with the usual list encoding. Clauses of width less than k can be expanded into the conjunction of their extensions on unused variables, using at most 2^k clauses when $m \geq k$.

Proposition 14.1 (Conditional hardness in a specified range). *Assume nonuniform SETH. For every fixed $0 < \varepsilon < 1$, some fixed k gives the verifier family (44), with $n, s = \Theta_k(m^k)$, whose projections satisfy*

$$C(g) > 2^{(1-\varepsilon)m} \quad \text{for infinitely many } m. \quad (46)$$

At the same time $C(g) \leq 2^m \text{poly}(m)$ by enumeration.

Proof. Otherwise the projections would have circuits bounded by a constant multiple of $2^{(1-\varepsilon)m}$ at every length. The finitely many exceptions can be absorbed in the constant. This contradicts the stated hypothesis for the fixed k it supplies, using the mask encoding. $\square$

This gives evidence for an exponent close to the exhaustive-search exponent, deep below the n -input ceiling. It does *not* prove a bound within a polynomial factor of $2^m s$, or a sharp $2^{m-o(m)}$ lower bound for one fixed k : the value of k may depend on ε . It also does not prescribe the worst case at every triple (n, m, s) .

Paturi and Pudlák study exponential resource tradeoffs for circuit satisfiability under explicit assumptions [28]. Williams shows that certain general-purpose algorithmic improvements over exhaustive search would themselves imply major circuit lower bounds [36]. These results help explain the difficulty of generic elimination. Neither is an unconditional lower bound $C(\exists y f) \gtrsim 2^m C(f)$ for arbitrary verifiers.

15 Relation to prior work and the scope of the contributions

The mechanisms behind the proofs have substantial histories. The purpose of this comparison is to identify the quantitative conclusions and integrations proved here, while attributing the tools on which they rest. It is a statement-level comparison, not a claim that a keyword search establishes priority.

Elimination and symbolic ordinary inputs. Bucket elimination and its combinations with conditioning are established methods [11]. The generalized distributive law applies marginalization to products in a commutative semiring [2]. Taking that semiring to be Boolean functions of x , with pointwise OR and AND, explains why ordinary inputs can remain symbolic. Storing each function as a circuit pointer makes a semiring operation cost one gate. Thus symbolic elimination is an application of an existing principle. The work in Section 7 is its concrete B_2 implementation, consistency budget, cubic reduction, and exact exponential-rate accounting.

Knowledge compilation has stronger target restrictions. DNNF supports efficient existential forgetting [9]. Bounded-treewidth Boolean circuits can be compiled into structured deterministic DNNFs with singly exponential dependence on width [4]. Here “deterministic” means that alternatives at an OR gate are mutually exclusive. An ordinary deterministic Boolean circuit has no such restriction. Forgetting may lose that property while still yielding a perfectly valid target for this note. Results retaining stronger structure through quantification, such as those of Capelli and Mengel [7], therefore solve a different representation problem. Their costs and lower bounds are not lower bounds on unrestricted $C(g)$.

Full circuit treewidth, treewidth after deleting input vertices, and the consistency graph used here are also different parameters. Prior bounded-treewidth circuit-SAT work explicitly uses an input-deleted graph [21]. Morizumi’s structural simulation uses layered circuit width and gives size $2^{O((w+\log s)\log s)}$ [25, Theorem 4]; that width is not interchangeable with our κ or cutwidth.

Graph width and tensors. The pathwidth theorem supplying the $1/6$ coefficient is due to Fomin and Høie [14]. The conversion between pathwidth and subcubic cutwidth is also known through graph-search relations; an explicit modern restatement appears in the proof of Lemma 16 of Bodlaender et al. [6]. Our direct median proof makes the additive constant and resulting exponential base transparent. Width-controlled tensor contraction is established [23], including Boolean checking circuits and counting applications [17]. The specific quantitative chain here is

$$h + b + \kappa \leq q + 1, \quad N \leq 2\kappa, \quad w \leq N/6 + o(N), \quad C(g) \leq p + O(q2^w).$$

Together with enumeration and nonuniform synthesis, it yields the $1/5$ projection rate. None of the inspected statements asserts this exact combined formula. This is the boundary of the comparison, not a proof that no equivalent earlier result exists.

There are also direct antecedents for the elementary tensor steps. Biamonte et al. [5] expand each COPY tensor into two consistent choices and evaluate the resulting trees, obtaining a bound polynomial in the representation size times 2^c for c COPY tensors in their canonical representation. This is the mechanism behind the forest-cut baseline. Its mixed-input specialization here leaves ordinary inputs symbolic and gives $r \leq s + 1 - a - b$. de Beaudrap et al. [10] explicitly switch between decision and counting semirings and study exact tensor rewrites. Kourtis et al. [18] give planar contraction guarantees and empirical studies of general cubic instances. The latter experiments do not supply a worst-case gate-count exponent. Dudek et al. [12] give exact weighted-counting tensor representations and tree factorizations of suitable high-degree tensors. Maximum intermediate rank controls storage; our sequential frontier calculation separately charges the operations needed for the time bound. These sources reinforce that the width-and-cycle accounting, rather than the general use of tensors or COPY expansion, is the quantitative step to assess here.

Appendix A likewise uses established polynomial identity testing and algebraic matching. Its transfer statement makes precise how a cheaper weighted evaluator would imply a smaller exact projection circuit, while unary tags preserve the cycle budget. It is an avenue for improving the main bound, not an additional improvement of its exponent.

Coverage, penalties, and sparse correction. The function $H \mapsto \sum_v N_v(H) + N_{\text{out}}(H)$ is a monotone submodular cost: each term counts covered restriction keys. Selecting witness columns with penalties for uncovered positive rows therefore fits an existing submodular-cost set-cover framework [34]. No approximation guarantee from that framework is imported here, and an algorithm polynomial in an explicit incidence matrix need not be polynomial in a succinct verifier circuit.

Optimizing only minimum row mass has an elementary fractional-cover interpretation. If g is nonzero and τ^* is its fractional witness-cover number, then

$$\max_{\mu} \min_{x: g(x)=1} \mu(W_x) = 1/\tau^*.$$

Indeed, divide a distribution by its minimum row mass to obtain a fractional cover, and normalize a fractional cover by its total weight for the converse. The occupancy objective additionally prices collisions on internal supports. The examples in Section 11.2 prove that even identical row-mass vectors leave an exponential ambiguity in that price.

Circuit synthesis for functions with few positive inputs is likewise classical; see Redkin [29, 30]. The block construction in Lemma 10.2 is a convenient all-range majorant, not a uniformly optimal sparse-synthesis theorem. In particular, the cited literature gives linear-size bounds in some ranges with a logarithmic number of positive inputs. The contribution of Theorem 10.4 is the joint inequality using the same sample for shared local computation and exact sparse correction, together with the distribution tradeoffs and hardness consequences proved from it.

The uniform counting consequence and its comparison. Corollary 7.8 makes the all-quantified consequence explicit. It proves count-preserving reductions, polynomial bit costs, and a fixed- ε running-time bound with exponential rate approaching $1/4$ in gate count. This uniform consequence is separate from the synthesis step in the nonuniform projection theorem. Its proof does not establish a claim to a previously best SAT algorithm.

The inspected branching literature records Nurk’s general Circuit-SAT bound $O(2^{0.4058s})$ [27] and Savinov’s improvement to $O(2^{0.389667s})$, presented by Lialina [20], and the nontrivial B_2 counting threshold below $3n$ gates of Golovnev et al. [15]. Width-based dynamic programming uses exponential space, whereas depth-first branching can have a different space cost; width-based SAT time–space tradeoffs are studied explicitly by Allender et al. [3]. The differing space costs do not, by themselves, establish whether the numerical consequence is new or explain its absence from particular benchmark discussions. For sufficiently small fixed ε , the proved counting exponent is smaller than these particular displayed decision exponents. This arithmetic comparison does not establish historical priority or the best unrestricted-space running time. The precise combined result may be a previously known consequence of width-based methods; the source comparison has not resolved that question.

What is established here. The displayed proofs establish the consistency-graph simulation and its $1/5$ rate, the necessary critical structure, the exact kernel realization with nonminimal verifiers, the occupancy–alteration bound, the quantitative sparse-fiber population, the all-quantified counting consequence, and the worked separations between compilation guarantees. The realization begins with the classical st -numbering property and separately proves its repeated-input repair budget and exact retained graph. It does not make a claim about the kernels of minimum verifiers. The ingredients attributed above are established prior methods. Priority for the precise combined statements is unestablished beyond the documented comparison. No unrestricted exponential lower bound, exact optimal exponent, or resolution of the remaining hard range is claimed.

16 What would sharpen the boundary?

The bounds give an explicit necessary window for a blowup close to exhaustive expansion. Stating the permitted loss avoids ambiguity about the meaning of “approximately $2^m s$.”

Proposition 16.1 (A necessary window, not a matching lower bound). *Let $s = C(f)$ and $L \geq 1$. If $C(g) \geq 2^m(s + 1)/L$, then, for every fixed $\varepsilon > 0$, there is a constant $A_\varepsilon \geq 1$ such that*

$$\boxed{\frac{m - \log(A_\varepsilon L)}{1/5 + \varepsilon} \leq s + 1 \leq \frac{LM_n}{2^m}.} \quad (47)$$

If $K = \min_{x: g(x)=1} |W_x|$, then also $K = O(nL)$.

Proof. Compare the assumed lower bound with $C(g) \leq M_n$ and with $C(g) \leq A_\varepsilon(s + 1)2^{(1/5+\varepsilon)(s+1)}$ from Theorem 7.6. Rearranging gives (47). The density conclusion follows by comparing with Proposition 10.1. $\square$

For any family with $m \rightarrow \infty$ and $\log L = o(m)$, the left inequality implies $\liminf(s+1)/m \geq 5$: first hold ε fixed and then let it tend to zero. A constant-factor match therefore needs $s + 1 \geq (5 - o(1))m$ as well as $s + 1 = O(2^{n-m}/n)$. These are necessary conditions in our gate model. They are not known to be sufficient. Corollary 10.7 strengthens the minimum-fiber condition to a population

bound: amplification of size $T = 2^m(s+1)/L$ requires at least $\Omega((T/n) \log(2 + T/n))$ positive inputs with $O(nL)$ witnesses.

The results above give several independent reasons for projection to be easy. For a chosen verifier circuit of size s , they can be combined as

$$C(g) \leq \min\{M_n, 2^m(s+1) - 1, \tau(f)s + \max\{\tau(f) - 1, 0\}, \sum_v 2^{|Y_v|} + 2^{|Y_{\text{out}}|} - 1, A(s+1)2^r, A_\varepsilon(s+1)2^{(1/5+\varepsilon)(s+1)}\}, \quad (48)$$

where A is an absolute constant, A_ε depends only on ε , and the structural terms use a normalized nontrivial output cone as in Section 6. Simple outputs are handled directly. The first, second, and last terms give bounds using n, m, s alone; the other terms use properties of the particular verifier. Two more informative bounds retain its internal geometry:

$$C(g) \leq p + O_\delta\left((q+1)2^{\min\{h, b, (1/3+\delta)\kappa\}}\right), \quad C(g) \leq B_\mu(t) + O\left(\frac{nE_\mu(t)}{\log(E_\mu(t) + 2)}\right).$$

The first exposes how ordinary preprocessing, witness count, and consistency cycles compete. The second couples reuse with the cost of repairing uncovered inputs. Both can be much smaller than a bound that retains only the total size and witness count.

Open question 16.2 (The remaining worst-case range). For specified parameter families with $m = \omega(\log n)$, s sufficiently large compared with m , and $2^m s \ll 2^n/n$, how large can $D(n, m, s)$ be? Can one obtain a general upper bound substantially below $2^m s$, or establish a quantitatively matching lower bound under a clearly stated hypothesis?

Three distinctions are useful in pursuing this question.

1. **A parameter region where hardness is possible is not a proved hard region.** The ceiling and the size-dependent simulation give necessary restrictions. They do not supply hard functions.
2. **A difficult witness cover is not a circuit lower bound.** A lower bound on $\tau(f)$ obstructs enumeration of a fixed witness set. A circuit for g may use a completely different representation. Likewise, large syntactic witness supports or many shared vertices need not force expensive projection.
3. **Exponential rate and multiplicative accuracy differ.** A lower bound $2^{(1-\varepsilon)m}$ is exponentially smaller than 2^m . A claim such as $\widetilde{\Omega}(2^m s)$ must specify which polynomial or logarithmic factors are being hidden and in which parameters. Witness padding must also be excluded as an explanation of the claimed ratio.

Exploit the gate tables beyond their graph. The $1/5$ proof uses a width bound for arbitrary cubic graphs. It ignores extra algebraic structure in tensors obtained from deterministic gate equations. Theorem 9.2 identifies the conditions that hypothetical amplification at the five-gate boundary must satisfy. Theorem 9.3 shows that acyclicity, essential inputs, and nearly critical supplied budgets still allow every simple 2-connected cubic kernel, even with an easy projection. Thus those conditions alone do not exclude such topologies. The realization uses a nonminimal verifier; restrictions specific to minimum circuits remain a separate possibility. A sharper contraction or simplification theorem that exploits gate semantics is a concrete route to a smaller exponent. The transfer is

explicit: if the relevant cubic kernels admit cutwidth $cN + o(N)$, the same proof replaces the limiting projection and counting coefficients by, respectively,

$$\frac{2c}{1+4c} \quad \text{and} \quad \frac{2c}{1+2c}.$$

Indeed $N \leq 2\kappa$ gives cycle exponent $2c\kappa + o(\kappa)$; balance against both h, b for projection and against b alone for counting. These are conditional transfer formulas, not improved width theorems. A uniform algorithm additionally needs suitable layouts to be constructible within the claimed running time.

Optimize a distribution together with its circuit representation. The worked families show two distinct losses in a density-only theory: identical row masses can hide exponentially different occupancies, and the unique best minimum-row-mass distribution can be worse for the joint objective. A natural next theorem would approximate or bound

$$\inf_{\mu, t \geq 0} \left[B_{\mu}(t) + A \frac{nE_{\mu}(t)}{\log(E_{\mu}(t) + 2)} \right]$$

for a fixed synthesis constant A , with integer t , on a useful class of succinct verifiers. One should also compare equivalent verifier circuits of comparable size, since moving a witness gate within a parity chain changes the occupancy objective. The target is an improved simulation theorem; a lower bound on this objective alone would still not lower-bound unrestricted $C(g)$.

Separate time savings from space savings and from hardness. The counting corollary makes a specific follow-up possible: determine whether its cycle-budget saving can be retained with substantially less space, and reconcile its exact statement with the older circuit-SAT literature. Separately, the sparse-fiber population theorem suggests studying the entire low-multiplicity tail in a concrete conditional hardness family. A successful hardness argument must add an ingredient that obstructs arbitrary circuits, rather than merely obstructing a witness cover or a chosen contraction. The general amplification claim and a matching description of $D(n, m, s)$ remain open.

A Weaker aggregation: a further direction

Exact integer counting supplies more information than existential projection needs. However, the earlier compiler already works over Boolean OR and AND. Changing its integer entries to bits alone changes the polynomial overhead, not the width exponent. A useful alternative aggregate must enable a different computation. The following standard polynomial-identity-testing route makes that target precise.

Discard unused inputs and write a, b for the used ordinary and witness counts, with $b \geq 1$. Over a field F of characteristic two, define

$$P_x(Z_1, \dots, Z_b) = \sum_{y \in \{0,1\}^b} f(x, y) \prod_{i: y_i=1} Z_i.$$

Distinct witnesses contribute distinct monomials, each with coefficient one. Hence P_x is the zero polynomial exactly when $g(x) = 0$, and its total degree is at most b . In contrast, unweighted parity can vanish on a nonempty fiber: the two witnesses 00, 01 contribute $1 + Z_2$, which vanishes at $Z_2 = 1$ but is not the zero polynomial.

Proposition A.1 (Fingerprint transfer to an exact circuit). *Let $F = \mathbb{F}_{2^\lambda}$ have size $Q > b$. Suppose a Boolean circuit of size T evaluates the λ -bit field value $P_x(z)$ from x and $z \in F^b$. If an integer $t \geq 1$ satisfies*

$$2^a(b/Q)^t < 1,$$

then

$$C(g) \leq t(T + \lambda) - 1.$$

In particular, $Q \geq 2b$ permits $t = a + 1$. If a division-free arithmetic circuit of A additions and multiplications evaluates $P_x(z)$, then $T = O((A + 1)\lambda^2)$ suffices.

Proof. The polynomial zero bound gives $\Pr_z[P_x(z) = 0] \leq b/Q$ whenever P_x is nonzero [31]. For completeness, a nonzero polynomial of total degree d has this bound d/Q : view it as a univariate polynomial of degree j in the last variable. Its leading coefficient has degree at most $d - j$; by induction it vanishes with probability at most $(d - j)/Q$. Otherwise the univariate polynomial has at most j roots. The union bound proves the claim.

Choose t independent evaluation points. For any positive input, the probability that all evaluations vanish is at most $(b/Q)^t$. A union bound over at most 2^a inputs proves that some fixed list works for every x . Hardwire that list. OR the λ output bits of each evaluation and then OR the t results, costing $tT + t(\lambda - 1) + (t - 1)$ gates. A negative input always gives zero, so the resulting circuit is exact on both kinds of input.

In a fixed polynomial basis for F , addition costs $O(\lambda)$ binary XOR gates. Schoolbook polynomial multiplication followed by reduction modulo a fixed irreducible polynomial costs $O(\lambda^2)$ gates. Field constants are fixed bit strings. This proves the arithmetic-circuit conversion. $\square$

The fixed list depends on the verifier; finding it is not part of the nonuniform conclusion. For a given input, random evaluations instead give a uniform one-sided error test when evaluation is uniformly efficient. For $Q \geq 2b$, the same argument uses $t = h + 1$ when only the h formal independent signals enter the evaluator, with their p preprocessing gates shared. Thus a faster field evaluator would transfer to a faster exact projection circuit with only polynomial overhead in the used-input counts.

The tags preserve the cycle budget. The consistency construction evaluates P_x if it is interpreted over F and each witness variable receives a unary weight equal to 1 at zero and Z_i at one. Its gate-value and equality extensions remain unique. Attaching a unary weight tensor adds one vertex and one edge, preserving the cycle rank. Equality splitting and the exact local contractions remain valid over F . Consequently weighting does not worsen the cycle exponent. Reusing the existing contraction order merely recovers that exponent; an improvement requires an algebraic evaluation method that does less work.

This differs from a generic implementation of witness isolation by dense affine constraints, as in the classical randomized isolation approach [33]. Directly adding k dense parity checks can cost $O(kb)$ gates. Any fine-grained use of that construction must account for the actual added gates and consistency constraints. Unary tags impose no such increase in cycle rank.

Proposition A.2 (No short universal linear fingerprint). *Fix a map $w : \{0, 1\}^b \rightarrow \mathbb{F}_2^L$, independently of a relation. If $\sum_{y \in W} w(y) \neq 0$ for every nonempty $W \subseteq \{0, 1\}^b$, then $L \geq 2^b$.*

Proof. The 2^b vectors $w(y)$ must be linearly independent over $\mathbb{F}_2$. Otherwise a nonzero dependence, whose coefficients are zeros and ones, supplies a nonempty zero-sum subset. $\square$

Concatenating several field evaluations is also such a binary linear fingerprint. The preceding transfer avoids this obstruction because its list is chosen for one verifier’s at most 2^a fibers, not for all subsets of the witness universe. This is a limitation of a fixed universal linear aggregate, not a circuit lower bound or a lower bound for a restricted class of small verifiers.

A classical example where the weaker aggregate changes the algorithm. Let $X = (x_{ij})$ encode a bipartite graph on $r + r$ vertices, and let the witness be a permutation σ certifying a perfect matching. The verifier checks distinct column choices and $\bigwedge_i x_{i,\sigma(i)}$. Binary column choices give $b = O(r \log r)$ witness bits and an $O(r^2 \log r)$ -gate verifier: compare every pair of columns and use a multiplexer in each row. Out-of-range column codes are rejected.

Assign a separate tag Z_{ij} to each potential edge. In characteristic two,

$$\det(x_{ij}Z_{ij}) = \sum_{\sigma \in S_r} \prod_i x_{i,\sigma(i)} Z_{i,\sigma(i)}.$$

All permutation signs equal one. Distinct matchings give distinct monomials, so the polynomial is nonzero exactly when a perfect matching exists. Its degree is r . Determinants over a finite field have polynomial-size Boolean evaluation circuits: perform Gaussian elimination with explicit zero-pivot branches and compute nonzero inverses by repeated squaring. All loop lengths are polynomial in r and $\log Q$. Applying the proof of Proposition A.1 with degree r , $a = r^2$, $Q \geq 2r$, and $t = r^2 + 1$ gives an exact polynomial-size projection circuit.

This is the classical algebraic matching phenomenon [26], not a new matching algorithm. Exact integer counting gives the permanent, whose computation on zero-one matrices is #P-complete [32]. The example explains what a successful weaker aggregate can accomplish; it supplies no unconditional separation of polynomial time from #P. For arbitrary B_2 verifiers, we have not found a field evaluator that improves the proved 1/5 projection rate.

B AI-assisted methods, verification, and reproducibility

This manuscript was developed in a user-directed session with an OpenAI Codex agent, described by the session as based on GPT-6. The user set the mathematical objective, requested an editable L^AT_EX note, and then asked for substantive new theory integrated with a pedagogical account of prior results. The model label records the provided session identity; it is not an independent characterization of the deployed model or its training.

Tools and organization. The agent worked in a local macOS workspace with a zsh shell, local file editing and Python execution, web retrieval of primary papers, a L^AT_EX toolchain, and PDF rendering. It used an author–review–revise workflow with separate child-agent mandates for structural proofs, literature comparison, and adversarial examples. A preliminary reviewer critiqued the research decomposition. The root agent developed the coverage construction and integrated the manuscript; further reviewers examined the analytical proofs and actual implementations under independently framed prompts. The agents shared a filesystem but were assigned distinct authoring files. The initial research phase involved no publication, submission, or external correspondence. A subsequent user-directed publication pass reframed the manuscript around its SAT algorithm and assembled the manuscript, supporting corpus, and reproducibility workflow as the public understanding-nondeterminism repository.

A further user-directed expansion developed the manuscript as a deep dive from computation paths to the counting algorithm. A decomposition critic reviewed that plan before separate authors

drafted the foundations and worked tables. The root developed the low-excess and conditional-count consequences, then integrated the sections. Independent mandates reviewed mathematical accuracy, pedagogical coherence, source attribution, and the pinning and generation arguments. The revisions made the reading order follow the worked example and stated exact sampling's probability-one termination and expected-time contract explicitly.

This organization supplies additional opportunities to detect errors; it does not turn agreement between agents into mathematical evidence. The agents can share training-related blind spots. The mathematical justifications are the complete arguments in the text and the precise published theorems they invoke, rather than the reviewers' confidence.

How candidate results were developed. The initial note combined enumeration, Shannon–Lupanov synthesis, fixed witness covers, and a forest simulation obtained by fixing shared vertices. Two limitations guided the next phase. First, a total count of shared values ignores how they are arranged. This led to the consistency graph, degree reductions with explicit tables, a cubic graph, and the pathwidth-based improvement. Second, optimizing witness coverage separately from sharing charges for computations that can be reused. This led to selected-restriction occupancies, a correction for uncovered positive inputs, and a single expected-cost argument.

Each candidate was written with its model and quantifiers before integration. The literature comparison separated existing elimination, synthesis, and graph-width tools from the proposed numerical and distributional consequences. Explicit families then tested what the new bounds explain that the old guarantee expressions miss. Counterexamples prevented stronger but false interpretations, such as choosing a different optimal witness cover independently at each gate.

An earlier focused research phase examined the critical three-way balance. A finite stability inequality was proved before taking fixed-slack limits, yielding necessary input, cycle-loss, and width conditions. An independent construction then realized cubic kernels with nearly critical supplied budgets. Its AND-only function permitted exact minimum-size calculations, exposing the difference between the supplied representation and intrinsic verifier complexity. Separate analytical review checked the conflict budget, sink construction, equality splitting, and legal contractions. The final source follow-up retrieved Nurk's full Russian preprint and additional text of Broering–Lokam; it narrowed access gaps without establishing priority.

When experiments were useful. The user explicitly required that experiments be used only when useful and never as a substitute for analytical proof of an asymptotic theorem. The finite checks here serve two purposes: testing circuit-producing implementations against independent direct evaluation, and checking exact arithmetic in worked finite instances. There was no empirical search for a circuit lower bound, no fitted asymptotic exponent, and no inference of novelty from experiment counts.

The checks found two concrete implementation omissions. The sampled compiler initially deduplicated full witnesses but not their output restrictions; this could charge too many output OR gates. It now deduplicates those restrictions explicitly. A reviewer also found that the tensor compiler omitted the zero-gate witness-output case already handled in the proof. That branch and designated-input checks were added. These findings justify the checks' role without extrapolating their conclusions beyond their actual domains.

In the exhaustive circuit checks, an encoding consists of one or two topologically ordered binary truth-table gates, with repeated input pins allowed and the final gate designated as output. There is one ordinary and one witness input. This is the precise 9,280-case domain.

Artifact	Completed finite checks and their scope
Forest and distribution checks	All 9,280 circuit encodings described above; 2,000 seeded circuits and three consistency fixtures. Separately, the exact projection distribution for all 65,536 relations with $(n, m) = (2, 2)$.
Coverage compiler	362 sparse support sets, 896 circuit/sample pairs, and 140 exact rational occupancy and expected-exception cases.
Tensor compiler	Five zero-gate designated-input cases, the 9,280 small circuits, 600 seeded circuits, and 140 symbolic tensor networks exercising loops, parallel edges, equality splitting, and median ties.
Worked families	25,600 input/witness pairs, support counts for $k = 3, \dots, 10$, finite distribution comparisons, and 141 sparse correction instances.
Kernel realization	3,729 gate-list and graph constructions, 72,401 legal degree-one/two graph contractions, and exact retained labeled kernels. Seven representatives also received exhaustive verifier truth-table and symbolic projection checks.

These enumerations establish agreement of the tested constructions on their stated finite domains. They do not establish minimum circuit complexity except where a separate analytical lower bound is given. In particular, the tensor checker does not implement or test the Fomin–Høie asymptotic pathwidth theorem; it checks the local reductions and contraction on supplied decompositions. The $1/5$ rate follows analytically from that cited theorem and the proved conversion, not from the finite graphs tested. The tensor checker evaluates Boolean projections. The integer-counting corollary is justified analytically. The expanded walkthrough adds finite integer-table checks, while the search and generation validator uses explicit finite counting oracles. No implementation or timing study of the full asymptotic counting algorithm is claimed.

Two further retained suites support the pedagogical expansion. The walkthrough checks all 4,096 triples of binary gate tables on all eight input assignments for unique internal extension; all 27 partial pinnings of the running example; equality splitting at degrees three through six; all 256 pairs of binary 2×2 tables under joint parallel contraction; and exact loop and frontier fixtures. The query suite checks every Boolean relation on zero through three bits: 278 relations, 1,844 prefix partitions, and 1,061 satisfying assignments under rank and unrank. It also checks 1,617 invalid ranks or nonsolutions and exact rejection probabilities for solution counts one through sixteen. These are finite comparisons against explicit enumeration and rational arithmetic. They neither implement the general layout theorem nor establish asymptotic performance experimentally.

The new realization checker addresses a specific possible mismatch: a graph sketch might fail to be an equality splitting of the actual circuit’s incidence graph. It verifies that correspondence and the legality of the graph reductions. Its finite graph domain contains every perfect matching disjoint from a fixed Hamiltonian cycle on 4, 6, 8, 10, 12 labeled vertices, 42 specified bridged block chains, and 32 seeded 2-connected cubic graphs. The 3,729 graph checks are distinct from the seven semantic checks. The general realization and critical-structure theorems are justified by their analytical proofs.

The user’s suggestion of a cheaper aggregate led to the characteristic-two fingerprint investigation in Appendix A. It was assessed analytically: a polynomial zero bound, a simultaneous hardwiring argument, exact cycle accounting, and a linear-algebra obstruction suffice. Separate mathematical and source reviews checked this appendix. No additional experiment was needed, and no improved general exponent resulted.

Reproducibility. The manuscript and research notes are distributed under [CC BY 4.0](#); the code and build automation use the MIT license. The [public repository](#) contains the license texts, citation metadata, and automated checks.

The entry point is `main.tex`. It includes the files in `sections/`. The file `references.bib` has an individual entry for each cited work. The `research/` directory contains the full statement matrix, supporting derivations, examples, and scripts. The review findings are retained in `review-notes/`. Run

```
make check
make pdf
```

from the project root. The check scripts use only the Python standard library. The PDF uses the installed $\text{\LaTeX}$ packages declared in the source. Seeds and exact finite domains are recorded in the scripts. Compilation checks references and typography mechanically; rendered pages are inspected separately because a successful build does not establish legibility. The research record distinguishes completed checks from pending work and retains the unresolved literature comparison described in Section 15.

References

- [1] Divesh Aggarwal, Huck Bennett, Alexander Golovnev, and Noah Stephens-Davidowitz. Fine-grained hardness of CVP(P): Everything that we can prove (and nothing else). arXiv:1911.02440v1, 2019. URL <https://arxiv.org/abs/1911.02440v1>. Definition 2.9 states nonuniform SETH.
- [2] Srinivas M. Aji and Robert J. McEliece. The generalized distributive law. *IEEE Transactions on Information Theory*, 46(2):325–343, 2000. doi: 10.1109/18.825794. URL <https://authors.library.caltech.edu/records/swlpm-bwj40>.
- [3] Eric Allender, Shiteng Chen, Tiancheng Lou, Periklis A. Papakonstantinou, and Bangsheng Tang. Width-parameterized SAT: Time-space tradeoffs. *Theory of Computing*, 10(12):297–339, 2014. doi: 10.4086/toc.2014.v010a012. URL <https://theoryofcomputing.org/articles/v010a012/>.
- [4] Antoine Amarilli, Florent Capelli, Mikaël Monet, and Pierre Senellart. Connecting knowledge compilation classes and width parameters. *Theory of Computing Systems*, 64(5):861–914, 2020. doi: 10.1007/s00224-019-09930-2. URL <https://arxiv.org/abs/1811.02944v2>. Online 2019. The author version restores “and” in the title.
- [5] Jacob D. Biamonte, Jason Morton, and Jacob W. Turner. Tensor network contractions for #SAT. *Journal of Statistical Physics*, 160(5):1389–1404, 2015. doi: 10.1007/s10955-015-1276-z. URL <https://arxiv.org/abs/1405.7375v2>. Theorem numbering refers to arXiv:1405.7375v2.
- [6] Hans L. Bodlaender, Edouard Bonnet, Lars Jaffke, Dušan Knop, Paloma T. Lima, Martin Milanič, Sebastian Ordyniak, Sukanya Pandey, and Ondřej Suchý. Treewidth is NP-complete on cubic graphs. *Electronic Journal of Combinatorics*, 32(3):P3.36, 2025. doi: 10.37236/13205. URL <https://www.combinatorics.org/ojs/index.php/eljc/article/view/v32i3p36>.
- [7] Florent Capelli and Stefan Mengel. Tractable QBF by knowledge compilation. In *36th International Symposium on Theoretical Aspects of Computer Science (STACS 2019)*, volume 126 of *Leibniz International Proceedings in Informatics*, pages 18:1–18:16, 2019. doi: 10.4230/LIPIcs.STACS.2019.18. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2019.18>.
- [8] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158. Association for Computing Machinery, 1971. doi: 10.1145/800157.805047. URL <https://www.cs.cmu.edu/~15455/>

- [resources/Cook1971-complx-thm-proof.pdf](#). Primary article transcription; Theorem 1 proof and Section 2.
- [9] Adnan Darwiche and Pierre Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002. doi: 10.1613/jair.989. URL <https://arxiv.org/abs/1106.1819>.
 - [10] Niel de Beaudrap, Aleks Kissinger, and Konstantinos Meichanetzidis. Tensor network rewriting strategies for satisfiability and counting. *Electronic Proceedings in Theoretical Computer Science*, 340:46–59, 2021. doi: 10.4204/EPTCS.340.3. URL <https://arxiv.org/abs/2004.06455v2>. Proceedings of QPL 2020.
 - [11] Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1–2):41–85, 1999. doi: 10.1016/S0004-3702(99)00059-4. URL <https://ics.uci.edu/~csp/r76A.pdf>.
 - [12] Jeffrey M. Dudek, Leonardo Dueñas-Orsorio, and Moshe Y. Vardi. Efficient contraction of large tensor networks for weighted model counting through graph decompositions. arXiv:1908.04381v2, 2020. URL <https://arxiv.org/abs/1908.04381v2>. Version 2, April 27, 2020; first posted 2019.
 - [13] Shimon Even and Robert Endre Tarjan. Computing an st -numbering. *Theoretical Computer Science*, 2(3):339–344, 1976. doi: 10.1016/0304-3975(76)90086-4. URL <https://www.sciencedirect.com/science/article/pii/0304397576900864>.
 - [14] Fedor V. Fomin and Kjartan Høie. Pathwidth of cubic graphs and exact algorithms. *Information Processing Letters*, 97(5):191–196, 2006. doi: 10.1016/j.ipl.2005.10.012. URL <https://fedorvf.github.io/articles/2006/2006b.pdf>. Theorem 5 and Section 4.
 - [15] Alexander Golovnev, Alexander S. Kulikov, Alexander V. Smal, and Suguru Tamaki. Gate elimination: Circuit size lower bounds and #SAT upper bounds. *Theoretical Computer Science*, 719:46–63, 2018. doi: 10.1016/j.tcs.2017.11.008. URL <https://eccc.weizmann.ac.il/report/2016/022/revision/2/download>. Expanded MFCS 2016 paper; inspected ECCC TR16-022 revision 2, December 1, 2017, which corrects Lemma 11.
 - [16] Mark R. Jerrum, Leslie G. Valiant, and Vijay V. Vazirani. Random generation of combinatorial structures from a uniform distribution. *Theoretical Computer Science*, 43:169–188, 1986. doi: 10.1016/0304-3975(86)90174-X. URL <https://ics.uci.edu/~vazirani/JVV.pdf>. Section 2 and Theorem 3.3; the source generator convention permits failure.
 - [17] T. H. Johnson, J. D. Biamonte, S. R. Clark, and D. Jaksch. Solving search problems by strongly simulating quantum circuits. *Scientific Reports*, 3:1235, 2013. doi: 10.1038/srep01235. URL <https://www.nature.com/articles/srep01235>.
 - [18] Stefanos Kourtis, Claudio Chamon, Eduardo R. Mucciolo, and Andrei E. Ruckenstein. Fast counting with tensor networks. *SciPost Physics*, 7(5):060, 2019. doi: 10.21468/SciPostPhys.7.5.060. URL <https://arxiv.org/abs/1805.00475v2>.
 - [19] Leonid A. Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266, 1973. URL <https://www.mathnet.ru/eng/ppi914>. English translation; Russian original in Problemy Peredachi Informatsii 9(3), 115–116.
 - [20] A. A. Lialina. On the complexity of unique circuit SAT. *Journal of Mathematical Sciences*, 247(3):457–466, 2020. doi: 10.1007/s10958-020-04813-1. URL <https://www.mathnet.ru/eng/zns16688>. Original POMI publication: Zap. Nauchn. Sem. POMI 475, 122–136 (2018). Includes Savinov’s general Circuit-SAT algorithm.
 - [21] Daniel Lokshtanov, Ivan Mikhailin, Ramamohan Paturi, and Pavel Pudlák. Beating brute force for (quantified) satisfiability of circuits of bounded treewidth. In *Proceedings of the*

- Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 247–261, 2018. doi: 10.1137/1.9781611975031.18. URL <https://sites.cs.ucsb.edu/~daniello/papers/boundedTreewidthCircuitSatSODA18.pdf>.
- [22] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya Vysshikh Uchebnykh Zavedenii. Radiofizika*, 1(1):120–140, 1958. URL <https://radiophysics.unn.ru/issues/1958/1/120>. In Russian; original title: Ob odnom metode sinteza skhem.
- [23] Igor L. Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008. doi: 10.1137/050644756. URL <https://arxiv.org/abs/quant-ph/0511069>.
- [24] Hiroki Morizumi. Lower bounds for the size of nondeterministic circuits. In *Computing and Combinatorics (COCOON 2015)*, volume 9198 of *Lecture Notes in Computer Science*, pages 289–296. Springer, 2015. doi: 10.1007/978-3-319-21398-9_23. URL <https://arxiv.org/abs/1504.06731v1>. Inspected author preprint arXiv:1504.06731v1, Theorem 2.
- [25] Hiroki Morizumi. Some results on the circuit complexity of bounded width circuits and nondeterministic circuits. arXiv:1811.01347v2, 2019. URL <https://arxiv.org/abs/1811.01347v2>. First posted in 2018; version 2, April 12, 2019. Theorem 3 and Section 4.
- [26] Ketan Mulmuley, Umesh V. Vazirani, and Vijay V. Vazirani. Matching is as easy as matrix inversion. *Combinatorica*, 7(1):105–113, 1987. doi: 10.1007/BF02579206. URL <https://people.eecs.berkeley.edu/~vazirani/pubs/matching.pdf>. Conference version: STOC 1987, 345–354, DOI 10.1145/28395.383347.
- [27] Sergey Nurk. An $O(2^{0.4058m})$ upper bound for circuit SAT. POMI preprint 10/2009, 2009. URL <http://www.pdmi.ras.ru/preprint/2009/09-10.html>. Full Russian preprint inspected; Theorem 3.1.
- [28] Ramamohan Paturi and Pavel Pudlák. On the complexity of circuit satisfiability. In *Proceedings of the 42nd ACM Symposium on Theory of Computing*, pages 241–250. ACM, 2010. URL https://cseweb.ucsd.edu/~paturi/myPapers/pubs/PaturiPudlak_2010_stoc.pdf.
- [29] N. P. Redkin. On the complexity of Boolean functions with small number of ones. *Discrete Mathematics and Applications*, 14(6):619–630, 2004. doi: 10.1515/1569392043272511. URL <https://www.mathnet.ru/eng/dm172>. Russian original: Diskretnaya Matematika 16(4), 20–31 (2004).
- [30] N. P. Redkin. Implementation complexity of Boolean functions with a small number of ones. *Discrete Mathematics and Applications*, 31(4):271–279, 2021. doi: 10.1515/dma-2021-0024. URL <https://www.mathnet.ru/eng/dm1592>. Russian original: Diskretnaya Matematika 32(2), 32–43 (2020).
- [31] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM*, 27(4):701–717, 1980. doi: 10.1145/322217.322225. URL <https://doi.org/10.1145/322217.322225>.
- [32] Leslie G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2): 189–201, 1979. doi: 10.1016/0304-3975(79)90044-6. URL https://www.cs.bu.edu/faculty/gacs/courses/cs535/papers/Valiant_permanent.pdf.
- [33] Leslie G. Valiant and Vijay V. Vazirani. NP is as easy as detecting unique solutions. *Theoretical Computer Science*, 47:85–93, 1986. doi: 10.1016/0304-3975(86)90135-0. URL <https://www.cs.toronto.edu/tss/files/papers/1-s2.0-0304397586901350-main.pdf>.
- [34] Fengmin Wang, Dachuan Xu, Donglei Du, and Chenchen Wu. Primal-dual approximation algorithms for submodular cost set cover problems with linear/submodular penalties. *Nu-*

- merical Algebra, Control and Optimization*, 5(2):91–100, 2015. doi: 10.3934/naco.2015.5.91. URL <https://www.aims sciences.org/article/doi/10.3934/naco.2015.5.91>.
- [35] Ingo Wegener. *The Complexity of Boolean Functions*. Wiley-Teubner Series in Computer Science. John Wiley and Sons and B. G. Teubner, 1987. ISBN 0-471-91555-6. URL https://web.vu.lt/mif/s.jukna/boolean/blaues_book.pdf. Chapter 4, Section 2: the Shannon effect for circuit size.
- [36] Ryan Williams. Improving exhaustive search implies superpolynomial lower bounds. In *Proceedings of the 42nd ACM Symposium on Theory of Computing*, pages 231–240. ACM, 2010. URL <https://people.csail.mit.edu/rrw/improved-algs-lbs2.pdf>.